DELTA-V TYCOON

Team Brief 2 of 6 · Version 1.1 (13 Nov 2026)

Programmer A: Simulation Core & Online Rooms

Formulas, checks, mission simulation, events, scoring, explanations, share codes, Firebase multiplayer

NASA Space Apps Cairo 2026 · Challenge: Space Mission Design Game
Hackathon: Fri 13 Nov 1:00 PM → Sat 14 Nov 1:00 PM (Cairo time)

This brief contains	Part B: your role, tasks, order of work and acceptance criteria. Part A: the shared project bible (identical in all six briefs): game design, every data value, every formula, file names and code contracts.
How to use it	Read Part B first, then Part A. Keep Part A open while you work. If Part A and Part B disagree, Part A wins. The brief is self-contained: hand both parts to any collaborator or tool and the work can be executed without extra context.
You own	src/core/**, src/net/**, src/data/index.ts, scripts/balance.ts, database.rules.json.
The six briefs	1 Team Lead · 2 Programmer A (core & rooms) · 3 Programmer B (UI & build) · 4 Character Artist · 5 Environment & Asset Artist · 6 QA & Playtest. Together they produce one game: every file in the repository has exactly one owner.
Golden rule	No production work before Friday 1:00 PM. Feature freeze Saturday 3:00 AM. Submit by Saturday 12:00 PM.

Contents

PART B — PROGRAMMER A: SIMULATION CORE & ONLINE ROOMS . . . 3

B1. Your mission . . . 3
B2. Deliverables and deadlines . . . 3
B3. Before H0 (allowed: setup only) . . . 3
B4. Step by step . . . 4
B5. Quality bar and gotchas . . . 10
B6. Handoffs and communication . . . 10

PART A — SHARED PROJECT BIBLE (identical in all six briefs) . . . 11

A0. What changed in v1.1 (read this first) . . . 11
A1. How to use this document . . . 11
A2. Event facts and hard rules . . . 11
A3. The challenge, and how we answer every part of it . . . 12
A4. Game concept . . . 13
A5. Feature list (IDs, priority, owner) . . . 14
A6. Screens, routes and flow . . . 15
A7. Game roles and team mechanics . . . 15
A8. Game data (exact values) . . . 16
A9. Formulas (Programmer A implements exactly; QA verifies with A17) . . . 21
A10. Readiness checks (runChecks) . . . 23
A11. Mission simulation (createSimulation) . . . 24
A12. Scoring (scoreMission) — maximum 1000 . . . 24
A13. Online rooms (Firebase Realtime Database) . . . 25
A14. Tech stack, repository and code contracts . . . 26
A15. Art direction and the complete asset manifest . . . 30
A16. Timeline, communication and definition of done . . . 32
A17. Reference designs (expected values — tests and QA use these) . . . 33

PART B — PROGRAMMER A: SIMULATION CORE & ONLINE ROOMS

B1. Your mission

You build the **brain** of Delta-V Tycoon: every number the player sees, every check, the day-by-day mission simulation, the score, the explanations, the share codes, and the online multiplayer layer. Your code has **no React and no CSS**. Programmer B builds every screen on top of your functions. If your numbers are right, the game is scientifically valid (a judging criterion); if your room layer is solid, the team-play feature works on stage.

You own these folders/files (only you edit them): `src/core/**`, `src/net/**`, `src/data/index.ts`, `scripts/balance.ts`, `database.rules.json`.

You read (owned by others): `src/data/*.json` (Lead writes content following A8 and your types), `src/state/store.ts` (B).

Your contract with everyone: Part A §A9–A14. Implement the formulas exactly. Do not rename any type, field, function or ID. If you must change a contract, ask the Lead, who posts it in `#bible-changes`.

B2. Deliverables and deadlines

By	Deliverable	Unblocks
H1:30	`src/core/types.ts` exactly as A14.1 + `src/net/roomApi.ts` interfaces (A14.3) pushed	Lead (JSON), B (UI)
H2:30	`src/data/index.ts` loader + validation; `rng.ts`; `catalog.ts` lookups	everything
H4	`computeBudgets`, `runChecks`, `defaultDesign`, `isUnlocked`, `DEFAULT_SPLIT`; `createLocalRoom`	B's design screen
H6	`createSimulation` without events + `scoreMission` (basic) + unit tests for A17	first public build
H9	Events (all 10) + insights + try-next + `simulateAll` + share/challenge codes + `conceptMaturity` + `findMissionTwin`	report + review
H9	Firebase setup + `createOnlineRoom` / `joinOnlineRoom` working for 2 browsers	B's lobby
H11	Council, funds requests, sign-off reset, decisions sync, presence, chat; security rules deployed; `estimateScience` + `tradeSweep`	team mode, F-34
H12	Support QA's online go/no-go test (1 AM)	cutoff decision
H13	`scripts/balance.ts` report for Lead; host migration and leaderboard only if ahead	balance
H14 (3 AM)	**Feature freeze** — bug fixes only	

B3. Before H0 (allowed: setup only)

1. Install Node 20 LTS and Git. Make sure `npm --version` works.
2. Create the Firebase project (account setup is allowed before kickoff; write no project code):
   1. Go to the Firebase console → **Add project** → name `delta-v-tycoon` → disable Google Analytics (not needed).
   2. **Build → Authentication → Get started → Sign-in method → Anonymous → Enable.**
   3. **Build → Realtime Database → Create database** → location **europe-west1 (Belgium)** (closest to Cairo) → start in **locked mode**.
   4. **Project settings → General → Your apps → Web (</>)** → register app `delta-v-tycoon-web` → copy the `firebaseConfig` object somewhere safe (you paste it at H8). These web keys are not secrets; they are safe to commit. Security comes from the database rules.
   5. **Authentication → Settings → Authorized domains:** add `<github-username>.github.io` (the Lead's account that owns the repo). `localhost` is already there.
3. Read Part A twice. Your formulas are in A9, checks in A10, simulation in A11, scoring in A12, rooms in A13, types in A14.

B4. Step by step

Step 1 (H0–H1:30): types and interfaces

- Create `src/core/types.ts` by copying A14.1 **exactly**. Add a `Catalog` interface:

```
export interface Catalog {
  orbits: Orbit[]; launchVehicles: LaunchVehicle[]; components: Components; instruments: Instrument[];
  scenarios: Scenario[]; difficulties: Difficulty[]; events: GameEvent[];
  glossary: GlossaryEntry[]; checksText: Record<string, { title: string; fail?: string; warn?: string; pass:
      string; fixHint: string }>;
  dialog: Record<string, DialogLine[]>; characters: Character[]; constants: Constants;
}
export interface GlossaryEntry { id: string; term: string; category: 'orbits' | 'rockets' | 'power' | 'data' |
      'spacecraft' | 'mission'; short: string; long: string; example: string; related: string[] }
export interface DialogLine { speaker: string; expr: 'neutral' | 'happy' | 'worried'; text: string; target?:
      string }
export interface Character { id: string; name: string; from: string; role: Role | 'npc'; playable: boolean;
      title: string; bio: string; lines: { success: string; failure: string; warning: string } }
export interface Constants { G0: number; SOLAR_EFFICIENCY: number; INTEGRATION_OVERHEAD: number; TANK_FRACTION:
      number; RADIO_STANDBY_W: number; MIN_BATTERY_WH: number; HGA_MAX_POINTING_DEG: number;
      DELAY_COST_M_PER_MONTH: number; BASE_DEV_MONTHS: number; ADVANCED_DEV_MONTHS: number; MARS_SYNODIC_MONTHS:
      number; ELECTRIC_MIN_W: number; ELECTRIC_DV_PER_MONTH: number; MAX_LIFETIME_YEARS: number; DAYS_PER_YEAR:
      number }
export interface SharePayload { v: 1; scenarioId: ScenarioId; difficulty: DifficultyId; seed: number; teamName:
      string; design: Design; split: BudgetSplit; decisions: Record<number, string> }
```

- Create `src/net/roomApi.ts` with the interfaces from A14.3 (no implementation yet). Push. Post "types pushed" in `#general`.

Step 2 (H1:30–H2:30): data loader, RNG, catalog lookups

`src/data/index.ts`: `import orbits from './orbits.json'` etc. for: orbits, launchVehicles, components, instruments, scenarios, difficulties, events, glossary, checks, dialog, characters, constants. Export `catalog: Catalog`. Add `validateCatalog(c)` that throws a readable error listing every problem:

- every id is unique inside its list;
- every `scenario.eventPool` id exists in events; every `defaultOptionId` exists in that event's options;
- every orbit has contactHours for all three networks; every scenario's `orbitFit` keys are valid orbit ids;
- every instrument tag used in `requiredTagGroups` exists on at least one instrument;
- every `checksText` key matches a check id from A10.

Call it once at startup in dev (`import.meta.env.DEV`). Until the Lead pushes JSON, create a tiny temporary JSON stub yourself in a branch so you are not blocked; delete it when the real files land.

`src/core/rng.ts`**:**

```
export function mulberry32(seed: number): () => number {
  let a = seed >>> 0;
  return () => {
    a = (a + 0x6D2B79F5) >>> 0;
    let t = a;
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
export function newSeed(): number { return (Math.random() * 2 ** 32) >>> 0; }
```

`src/core/catalog.ts`**:** `getCatalog()`, `getOrbit(id)`, `getVehicle(id)`, `getBus(id)`, `getArray(id)`, `getBattery(id)`, `getComputer(id)`, `getAdcs(id)`, `getPropulsion(id)`, `getRadio(id)`, `getGround(id)`, `getInstrument(id)`, `getScenario(id)`, `getDifficulty(id)`, `getEvent(id)`. Each throws `Unknown <kind> id: <id>` if missing. Also `allSelectableItems(design)` returning the selected item objects with their tier (used for `advancedCount` and `isUnlocked`).

Step 3 (H2:30–H4): budgets, checks, defaults

`budgets.ts` → `computeBudgets(design, scenarioId, difficulty, split)`**.** Follow A9 line by line. Notes:

- `solarArrayId === 'none'` → arrayW contributes 0 mass/cost/power.
- `x = exp(dv/(isp·G0)) − 1`; `den = 1 − TANK_FRACTION·x`; if `prop_none` → propellant 0, capability 0; if `den ≤ 0.05` → `feasible=false`, propellant = 0, capability = 0.

- `maxPropellantKg` cap (cold gas): cap the propellant, recompute capability with the log formula.
- Rideshare: `launchCapacityKg` = `capacityKg.SSO` only when orbit is sso; otherwise `null`.
- `launchMarginPct` is `null` when capacity is `null`.
- Fill the `lines` arrays for breakdown tooltips (label, value, role). Mass lines: Bus, each instrument, Solar array, RTGs, Battery, Computer, Attitude control, Propulsion (dry), Radio, High-gain antenna, Propellant, Tank. Power lines: same components with W. Cost lines: each hardware item ×1.2, Launch, Operations, Ground network, Delay.
- `time`: implement A9 exactly, including the Mars window loop.
- Round only for display (B's job). Keep full precision inside.

`checks.ts` → `runChecks(...)`**.** One function per check id in A10, all using the result of `computeBudgets`. Build the complete variable object listed in A10 ("Template variables") **once** per call. Take the title/message text from `catalog.checksText[id]` (`pass` / `warn` / `fail` by severity) and fill placeholders with `fill(template, vars)` where `{name}` is replaced by `vars.name` (numbers formatted with at most 1 decimal; money as `$12.5M`). `role_allocation` produces one check per role (`role_allocation_planner`, etc., all using the `role_allocation` text). Return sorted: fail → warn → pass, then by role order in `ROLES`.

`defaultDesign(scenarioId, difficulty)`**.** Starter designs are deliberately imperfect, so the first thing the player learns is to read the checks:

scenario	planner	engineer	science	comms
nile_watch	sso, 2 yr, spark, margin 10	bus_small, sa_si 300 W, bat_liion 200 Wh, obc_basic, adcs_wheels, prop_mono	imager_ms	radio_sband, no hga, campus
storm_watch	geo, 5 yr, workhorse, 10	bus_medium, sa_si 500, bat_liion 1000, obc_basic, adcs_wheels, prop_biprop	sounder_atm	radio_xband, no hga, commercial
ice_hunter	lunar, 1 yr, workhorse, 10	bus_medium, sa_si 500, bat_liion 400, obc_basic, adcs_wheels, prop_biprop	spec_neutron	radio_sband, no hga, deepspace
red_relay	mars, 2 yr, workhorse, 10	bus_medium, sa_si 1000, bat_liion 500, obc_basic, adcs_wheels, prop_biprop	spec_atm	radio_xband, no hga, deepspace
first_light	l2, 5 yr, workhorse, 10	bus_medium, sa_si 800, bat_liion 100, obc_basic, adcs_wheels, prop_mono	tel_ir	radio_xband, no hga, deepspace
sandbox	leo, 2 yr, spark, 10	bus_small, sa_si 300, bat_liion 200, obc_basic, adcs_mag, prop_mono	cam_wide	radio_sband, no hga, campus

Then replace any item that is not unlocked for the difficulty with the cheapest unlocked alternative of the same kind.

`isUnlocked(itemId, difficulty, scenarioId)` as specified under A14.2.

Step 4 (H4–H5): LocalRoom

`src/net/localRoom.ts`: an in-memory `RoomApi`. One local player `uid = 'local'` owns all four roles. State starts as: meta (code null, phase `council`, seed given or `newSeed()`, teamName = player name), council split `DEFAULT_SPLIT` with `agreed = {}`, design = `defaultDesign(...)`, signoffs all false, empty decisions/suggestions/chat/requests, result null. Every mutator updates the state immutably (new object) and notifies subscribers synchronously. Implement exactly the same behaviour rules as the online room (design update clears signoffs; setSplit clears agreed; answerRequest moves split points: `points = amountM / budgetM × 100`, never letting a role go below 0). This means B can build the whole UI before Firebase exists.

Step 5 (H5–H6): simulation (no events yet) and basic score

`sim.ts` → `createSimulation(input)` returns an object implementing `Simulation`. Internal state:

```
{ day, endDay, plannedDays, rng, stored, lost, dataGen, dataRet, sciGen, sciBonus, extraCost,
  modifiers: { type: 'scienceMult'|'powerMult'|'dataMult'|'relayMult'|'contactLoss'|'scienceOff', value: number,
      untilDay: number | null }[],
  pointingMult: number, batteryMult: number, groundNetworkId, lifetimeYears, dvRemaining,
  schedule: { index, day, eventId, roll }[], nextEventIdx, pending: PendingEvent | null,
  events: EventLogEntry[], samples: Sample[], outcome, outcomeReason, immune: Set<string> }
```

Algorithm (A11): run the launch and insertion checks at creation; if they fail, `done = true` at day 0 with the right outcome. `step(n)` loops day by day until `n` days have passed, an event is due (`schedule[nextEventIdx].day === day`), or `day === endDay`. Recompute only what changes per day (degradation, active modifiers); precompute everything constant from `computeBudgets` once. Recompute pointing quality when `pointingMult` changes (effective ADCS pointing = `adcs.pointingDeg × pointingMult`;

this also affects `hgaEffective` and therefore the downlink). Propellant: `dvRemaining` starts at `capability − insertionDv`; each day subtract `stationKeepingDvMsPerYear / 365`; when `dvRemaining ≤ disposalDvMs` the mission ends early with reason "out of propellant to maintain the orbit" (disposal reserve kept). `deltaVUse` effects subtract from `dvRemaining`. `lifetimeYearsDelta` changes `endDay` (never below the current day + 1).

`score.ts` → `scoreMission(input, result, team)` exactly per A12. `roleGoals` uses A7: planner = `finalTotalCost ≤ 0.9 × budget && delayMonths == 0`; engineer = `launchMarginPct ≥ 10 && powerMarginPct ≥ 15 && batteryOk`; science = `scienceReturned ≥ scienceTarget`; comms = `dataLost == 0 && dataReturned ≥ 0.95 × dataGenerated`. Mass and power margins for efficiency come from the design budgets (pre-launch). `finalReservePct` uses `finalTotalCostM` (planned cost + event costs).

Tests (`src/core/__tests__/budgets.test.ts`): one test per row of A17 checking total kg, launch margin, load, generation, battery required, downlink, return % and total cost within ±0.15 (rounding). These tests protect the team all night. Run with `npx vitest run`.

Step 6 (H6–H9): events, insights, share codes

`events.ts`:

- `scheduleEvents(scenario, difficulty, budgets, design, rng)`: count = `max(1, round(eventCount × eventMult))`. Filter the pool by `conditions` (orbits list, `adcsIn` list, `needsEclipse` → orbit.eclipseMin > 0). Weighted draw without replacement: weight 1, or `weightIfRadio.weight` if the design uses that radio. Draw days as in A11. Pre-draw `roll = rng()` for each scheduled event, in schedule order, right after the days.
- `buildPending(index, event, state, design, budgets)`: if `event.autoHandledByComputer === design.computerId`, auto-resolve: log an `autoHandledText` entry, apply no option, and continue without pausing. Otherwise apply `autoEffects`, then build the options list with `disabled` + `disabledReason` from `requires`:
  - `propulsion`: propulsion is not prop_none, and dvRemaining > the option's deltaVUse;
  - `nearEarth`: orbit.isNearEarth;
  - `busSpareWheel`: bus.hasSpareWheel;
  - `reservePctOfBudget`: current reserve ≥ that % of budget;
  - `autonomyIfDelayOverSec`: if orbit.lightDelaySec > value then computer.autonomy must be true. Reason text: "Signals take {minutes} minutes to reach the spacecraft — without onboard autonomy you cannot react in time."
  - If the default option is disabled, use `defaultOptionIdIfNoPropulsion` or the first enabled option.
- `applyOption(option, roll, state)`: apply effects; if `risk`, compute `chance` (use `chanceIfRule` when `chanceRule` is true: `power_margin_after_hit` = (generation now − load)/load < 0; `battery_below_required` = batteryWh × batteryMult < required), multiply every applicable reduction (`ifInstrument` present in design, `ifComputer` equal to computer id); if `roll < chance` apply risk effects and mark bad, text = `badExplanation`; else text = `explanation`. A chosen option with `good: false` and no risk triggered still counts as **not bad**. Only triggered risks, `endMission`, and `budget_cut → shorten` that drops the lifetime below the minimum count as bad.
- `budget_cut` amounts: `costM` value = 10% of budgetM (compute at runtime; JSON stores `value: 10` with `reason: "pct_of_budget"`).
- `scienceBonusPct`: adds `value/100 × (one year of science at full duty with current modifiers)` to `sciBonus`.
- `downgradeGround`: deepspace → commercial → campus; if the new network has 0 contact hours for this orbit, the option is disabled ("this orbit needs the deep-space network").
- `immunity`: add the event id to `immune`; skip later scheduled occurrences of it.

`insights.ts` → **used by** `scoreMission`**.** Each rule produces an `Insight`. Use these templates (numbers rounded; `{}` are values):

Rule	Kind	Role	Text
outcome launch_failure (too heavy)	bad	planner	Your spacecraft weighed {total} kg but the {vehicle} can only carry {cap} kg to this orbit. Choose a bigger rocket or remove mass.
outcome launch_failure (anomaly)	bad	planner	The {vehicle} failed during ascent. Rockets with {rel}% reliability fail about {fails} in 100 launches. A more reliable rocket lowers that risk.
outcome orbit_failure	bad	engineer	The spacecraft needed {ins} m/s to enter orbit but could only produce {cap} m/s. More propellant or a higher-Isp engine fixes this.

Rule	Kind	Role	Text
outcome power_failure	bad	engineer	On day {day} the solar arrays produced less power than the computer and radio need, and the spacecraft shut down. Size arrays with margin for aging ({deg}% per year).
ended_early propellant	bad	engineer	Propellant for keeping the orbit ran out after {years} of {planned} years. Increase the delta-v margin or choose a more efficient engine.
launch margin 5–40%	good	planner	Launch mass margin of {m}% left room for surprises without paying for a rocket that was too big.
launch margin < 5%	bad	planner	Only {m}% launch margin. Real missions grow in mass during development; keep at least 10%.
power margin ≥ 15%	good	engineer	{p}% power margin kept every instrument running even as the arrays aged.
duty reduced	bad	engineer	Instruments ran at {d}% on average because power was short. Bigger arrays or fewer power-hungry instruments would help.
battery not ok	bad	engineer	The battery could not carry the full load through each {e}-minute eclipse, so instruments were switched off in the shadow.
data return ≥ 95%	good	comms	{r}% of the data reached Earth. The link to {gs} kept up with the instruments.
data return < 95%	bad	comms	Only {r}% of the data reached Earth; {lost} Gbit was lost. Faster radio, a high-gain antenna, a bigger ground network, or a compression computer would fix it.
pointing quality < 1	bad	science	{instr} needs {req}° pointing but your attitude control holds {adcs}°, so its images were blurred to {q}% quality.
science ≥ target	good	science	Science return reached {s}% of the goal.
science < 70%	bad	science	Science return reached only {s}% of the goal. {Biggest cause from the other insights.}
missing required group	bad	science	The mission needed a {tags} instrument and none was on board, so the main objective could not be met.
orbit fit < 1	bad	planner	{orbit} works for this mission, but only at {fit}% effectiveness. {recommended orbit name} suits it best.
reserve ≥ 15%	good	planner	You finished with {res}% of the budget in reserve — that money can fund the next mission.
over budget	bad	planner	The mission cost {cost} against a budget of {budget}. Real agencies cancel missions that overrun.
debris rule violated	bad	planner	The spacecraft cannot deorbit at end of life, adding to space debris in a crowded orbit. Carry {disp} m/s for disposal.
each good event choice	good	owner role	{event title}: {explanation}
each bad event outcome	bad	owner role	{event title}: {badExplanation}

Try next: take the bad insights in this priority order: missing required group → launch/orbit/power failure → data return → duty/battery → pointing → science → budget. Map each to a concrete suggestion (one sentence that names a component, e.g. "Swap the S-Band Transmitter for the X-Band Transmitter (+3 kg, +25 W, +$2M) to multiply your downlink by 75."). Compute the suggested change's effect with `computeBudgets` on a modified design, so the numbers are real. Return at most 3.

`share.ts`**:** `encodeDesign` / `decodeDesign` with lz-string `compressToEncodedURIComponent` / `decompressFromEncodedURIComponent`; validate `v === 1` and ids exist, else return null. `encodeChallenge(s, d, seed)` = `` `DVT-${s}-${d}-${seed.toString(36)}` ``; `decode` with a regex `^DVT-([a-z_]+)-(cadet|engineer|director)-([0-9a-z]+)$`.

`simulateAll(input, decisions)`**:** create a simulation, step until done, answering each pending event with `decisions[index] ?? defaultOptionId`. Used for replays, leaderboard verification and balance.

Step 7 (H7–H11): online rooms with Firebase

Install: `npm i firebase lz-string`.

`src/net/firebase.ts`**:**

```
import { initializeApp } from 'firebase/app';
import { getAuth, signInAnonymously } from 'firebase/auth';
import { getDatabase } from 'firebase/database';
const firebaseConfig = { /* paste from console */ };
export const app = initializeApp(firebaseConfig);
export const auth = getAuth(app);
export const db = getDatabase(app);
export async function ensureSignedIn(): Promise<string> {
  if (auth.currentUser) return auth.currentUser.uid;
  const cred = await signInAnonymously(auth);
  return cred.user.uid;
}
```

Import this module **lazily** (`await import('./firebase')`) inside `createOnlineRoom` / `joinOnlineRoom`, so solo play never loads Firebase and still works offline.

`roomCodes.ts`**:** `makeCode()` = 5 random chars from the A13 alphabet; `normalizeCode(s)` = uppercase and strip spaces; return null if any character is outside the alphabet (B then shows "Check the code — it has 5 letters/numbers, no O, 0, I, 1 or L").

`firebaseRoom.ts` **— implementation notes:**

- **Create:** `uid = ensureSignedIn()`; loop up to 5 times: `code = makeCode()`; `runTransaction(ref(db, 'rooms/'+code+'/meta'), cur => cur === null ? meta : undefined)`; if committed, done. Then write `players/{uid}`, `council` with DEFAULT_SPLIT, `design` = defaultDesign, signoffs false, roles all null. Then claim all four roles for the creator? **No**: the creator claims none automatically. The lobby lets each player tap the roles they want, and the host can press "Fill empty roles with me".
- **Join:** read `rooms/{code}/meta` once (`get`). Missing → throw `ROOM_NOT_FOUND`. Count `players` with `connected: true`; if ≥ 4 and you are not already in the list → `ROOM_FULL`. Write own player record.
- **Presence:** `onValue(ref(db, '.info/connected'), snap => { if (snap.val()) { onDisconnect(playerRef).update({ connected: false, lastSeen: serverTimestamp() }); update(playerRef, { connected: true, lastSeen: serverTimestamp() }); } })`.
- **Subscribe:** one `onValue(ref(db, 'rooms/'+code), snap => ...)` listener; convert the raw value into a full `RoomState` with defaults (missing arrays → [], missing design parts → defaults, `requests` and `chat` objects → arrays sorted by `createdAt`/`at`, chat limited to the last 50). Cache the last state for `getState()`.
- **claimRole:** `runTransaction(ref(db, 'rooms/'+code+'/roles/'+role), cur => (cur === null || cur === uid) ? uid : undefined)` → return `committed`.
- **updateDesign(role, patch):** build one multi-path update: `{ ['design/'+role+'/'+key]: value ... , 'signoffs/planner': false, 'signoffs/engineer': false, 'signoffs/science': false, 'signoffs/comms': false, 'reviewDone': false }` → `update(ref(db, 'rooms/'+code), paths)`. Arrays (instrumentIds) are written whole.
- **setSplit:** update `council/split` and set `council/agreed` to null in the same multi-path update.
- **requestFunds:** `push(ref(..., 'requests'))` with status pending. **answerRequest:** transaction on `council/split` that moves the points (clamped), then set the request status. Only the owner of `toRole` may answer (check in the client).
- **submitDecision:** `runTransaction(ref(.., 'decisions/'+i), cur => cur === null ? {optionId, byUid: uid, at: Date.now()} : undefined)`.
- **sendChat:** `push` with text trimmed to 200 chars; ignore empty.
- **Host migration (P1):** every client checks on each state update: if the host's player record has `connected === false` and `Date.now() − lastSeen > 30000`, then the connected player with the smallest `joinedAt` runs a transaction on `meta/hostUid` setting it to their own uid (only if it still equals the old host).
- **Errors:** wrap every call; on network failure, reject with `Error('NETWORK')` so B can show "Connection lost — retrying". The Firebase SDK queues writes offline and resyncs automatically.

Ops synchronisation (agree this exact behaviour with B): every client creates its own `Simulation` from `{scenarioId, difficulty, seed, design, split}` when `phase` becomes `ops`. B's ops screen calls `sim.step(k)` per animation frame. When `sim.pendingEvent` appears, the UI stops stepping and waits for `state.decisions[pending.index]`. When it exists, call `sim.decide(optionId)` and continue. Because the simulation is deterministic and everyone uses the same decisions, every screen shows the same mission. The timer: when it reaches 0, any client calls `submitDecision(index, defaultOptionId)`; the transaction makes sure only the first write wins.

`database.rules.json` (deploy in console → Realtime Database → Rules, or with the Firebase CLI):

```
{
  "rules": {
    "rooms": {
      "$code": {
        ".read": "auth != null",
        "meta": {
          ".write": "auth != null && (!data.exists() || data.child('hostUid').val() === auth.uid ||
      (root.child('rooms/'+$code+'/players/'+auth.uid).exists() && newData.child('hostUid').val() === auth.uid))"
        },
        "players": { "$uid": { ".write": "auth != null && $uid === auth.uid" } },
        "roles": { "$role": {
          ".write": "auth != null && root.child('rooms/'+$code+'/players/'+auth.uid).exists() && (!data.exists()
      || data.val() === auth.uid || root.child('rooms/'+$code+'/meta/hostUid').val() === auth.uid)"
        } },
        "council":     { ".write": "auth != null && root.child('rooms/'+$code+'/players/'+auth.uid).exists()" },
        "requests":    { ".write": "auth != null && root.child('rooms/'+$code+'/players/'+auth.uid).exists()" },
        "design": { "$role": {
          ".write": "auth != null && (root.child('rooms/'+$code+'/roles/'+$role).val() === auth.uid ||
      (!data.exists() && root.child('rooms/'+$code+'/meta/hostUid').val() === auth.uid))"
        } },
        "signoffs":    { ".write": "auth != null && root.child('rooms/'+$code+'/players/'+auth.uid).exists()" },
        "reviewDone":  { ".write": "auth != null && root.child('rooms/'+$code+'/players/'+auth.uid).exists()" },
        "decisions": { "$i": { ".write": "auth != null && !data.exists() &&
      root.child('rooms/'+$code+'/players/'+auth.uid).exists()" } },
        "suggestions": { ".write": "auth != null && root.child('rooms/'+$code+'/players/'+auth.uid).exists()" },
        "chat": { "$m": {
          ".write": "auth != null && !data.exists() && root.child('rooms/'+$code+'/players/'+auth.uid).exists()",
          ".validate": "newData.child('text').isString() && newData.child('text').val().length <= 200"
        } },
        "result": { ".write": "auth != null && root.child('rooms/'+$code+'/meta/hostUid').val() === auth.uid" }
      }
    },
    "leaderboard": {
      ".read": "auth != null",
      "$scenario": { "$entry": {
        ".write": "auth != null && !data.exists()",
        ".validate": "newData.child('score').isNumber() && newData.child('score').val() <= 1000 &&
      newData.child('teamName').isString() && newData.child('teamName').val().length <= 24"
      } }
    }
  }
}
```

Important ordering detail: the room creator must write `meta` **before** `players/{uid}`, and `design/*` for the starter design right after (the host may write a design subtree only while it does not exist yet).

Step 7b (H9–H11): Concept Maturity, Mission Twin, Trade-Space Explorer

`maturity.ts` → `conceptMaturity(checks)` exactly as A10 "Concept Maturity meter": count pass/warn/fail; `greenPct = pass / total × 100`; CML 4 if greenPct ≥ 80 and fail = 0; CML 3 if greenPct ≥ 50; else CML 2. Names: 2 "Initial Feasibility", 3 "Trade Space", 4 "Point Design".

`twins.ts` → `findMissionTwin(design, scenarioId)`: candidates = `scenario.twins` (sandbox: all twins of all scenarios). Score = 2 if `twin.orbit === design.planner.orbitId`, + 1 per tag in `twin.keyTags` present among the design's instrument tags. Return the highest (first on ties); return null if the best score is 0.

`trade.ts` → `estimateScience(...)`: the science a design returns over its planned lifetime **with no events**, without running the day loop: `Σ_i baseScience_i × tagWeight_i × orbitFit × quality_i × duty0 × lifetimeYears × returnFraction`, where `duty0 = clamp((generationW − nonInstrumentLoad) / Σ instrument.powerW, 0, 1) × (batteryOk ? 1 : max(0.2, batteryWh / batteryRequiredWh))` at beginning of life, and `returnFraction = min(1, downlinkGbitPerDay / generatedGbitPerDay)` (1 if nothing is generated). If launch or insertion would fail (capacity, destination, delta-v), the estimate is 0.

`tradeSweep(design, scenarioId, difficulty, split, param)` (the JPL "science partials"): build a list of candidate values, apply each to a copy of the design, and return one `SweepPoint` per value from `computeBudgets` + `runChecks` + `estimateScience`. Values:

param	values
lifetimeYears	0.5 to 10, step 0.5
solarArrayW	0 to 3000 step 100 (or up to 6000 step 200 if the current value is above 3000)
batteryWh	50 to 3000 step 50
deltaVMarginPct	0 to 30 step 5

param	values
busId, radioId, groundNetworkId, launchVehicleId, adcsId	every unlocked option of that kind

`label` is human-readable ("X-Band Transmitter", "2.5 yr", "700 W"). Mark `isCurrent` for the design's present value. Must run in < 30 ms for 60 points: no simulation loop here.

Tests: maturity thresholds (79% green → CML 3; 80% with 1 red → CML 3; 80% with 0 red → CML 4); twin for A17 nile_watch design = Landsat 9; twin for A17 ice_hunter design = Lunar Reconnaissance Orbiter; `estimateScience` of the A17 nile_watch design ≈ 41 × 3 = 123 (±1); sweep returns `isCurrent` exactly once.

Step 8 (H11–H13): leaderboard, balance script

`leaderboard.ts`**:** local leaderboard in `localStorage['dvt.leaderboard']` (wrap every read/write in try/catch; on failure keep it in memory). Entry = `{ teamName, scenarioId, difficulty, seed, score, stars, outcome, designCode, at }`. `addLocal(entry)`, `listLocal(scenarioId?, challengeSeed?)`. Online (P2): `postOnline(entry)` → `push(ref(db, 'leaderboard/'+scenarioId))`; `listOnline(scenarioId)` → `query(..., orderByChild('score'), limitToLast(20))`. Before showing an online entry, verify it: `decodeDesign(designCode)` → `simulateAll` → `scoreMission`; if the recomputed score differs by more than 1, hide it.

`scripts/balance.ts` (run `npx tsx scripts/balance.ts`): for each scenario, run the A17 reference design and the default design over 200 seeds with default decisions, and also with "all good decisions"; print min / median / max score, % success, average stars. Also try 20 random valid designs per scenario and print the best 3 (proves several designs can win). Post the table in `#general` for the Lead at H13 (2 AM) and again after any balance change.

B5. Quality bar and gotchas

- **Determinism:** never call `Math.random()` inside core except `newSeed()`. Never depend on object key order for anything random. Iterate instruments in `design.science.instrumentIds` order.
- **Performance:** a 10-year mission is 3,650 days; `simulateAll` must finish in < 50 ms. Precompute constants once; the per-day loop should be plain arithmetic.
- **Numbers:** never round inside core. Provide `fmt` helpers in `src/core/format.ts` for B: `fmtKg`, `fmtW`, `fmtM` ($12.5M), `fmtPct`, `fmtGbit`, `fmtMonths`.
- **Edge cases to handle:** zero instruments (science 0, data 0 → return 100%); `solarArrayId 'none'` with RTGs; GEO with campus (24 h contact is fine); battery 0 Wh on an orbit with no eclipse; lifetime 0.5 years; budget cut making lifetime < current day; electric propulsion with low power; cold-gas propellant cap.
- **Units in UI text:** always write units ("kg", "W", "m/s", "Gbit/day", "$M").
- **Tests to have green at freeze:** A17 budgets (6 rows); determinism (same seed and decisions → identical result JSON); bad example produces insights "data return" and "pointing"; red_relay with basic parts and sounder_atm (devMonths 30) → launchMonth 52, delay cost 39; rideshare with only cam_wide (devMonths 18) → launchMonth 20; launch over capacity → launch_failure; prop_none in LEO with 5-year life → ended_early at 4 years; share code round trip; challenge code round trip.

B6. Handoffs and communication

- To **B:** post in `#general` each time a function lands: name, file, one example call. Answer B's questions within 10 minutes; B is your customer.
- To **Lead:** JSON field questions; the balance table at H13; any formula change request.
- To **QA:** list of "testable now" items with every push; QA's bug issues tagged `[A]` are yours.
- Ask a mentor (Software Engineering / Web Development track) early if Firebase blocks you for more than 30 minutes. Remember the cutoff rule (A16): solo and same-screen must never depend on Firebase.

PART A — SHARED PROJECT BIBLE (identical in all six briefs)

A0. What changed in v1.1 (read this first)

- **Official NASA resources integrated** (A3): JPL's *Concept Maturity Levels* paper, NASA's *Design and Develop Science Missions* webinar, NASA Webb's *Build It Yourself* game and NASA's *mission-viz* orbit visualizations. They now shape the game structure, not just the sources list.
- **New features:** F-33 Mission Twin (the real mission your design resembles), F-34 Trade-Space Explorer (how science changes when you change one parameter), F-35 Concept Maturity meter in the readiness review (CML 2 → 4), F-36 requirements with a "Verified by" column, NASA process labels on the phase stepper, a "Who uses your data" line (user segment), Kepler-ellipse orbit drawing and transfer animation.
- **New playable character: Diaa El Awdi**, a third Mission Commander (he agreed to appear). Character files: 33 portraits, 11 idle sprites, 9 suit sprites.
- **New schedule: feature freeze at 3:00 AM Saturday (H14).** Everything after 3 AM is bug fixes, footage, video, slides and submission. All deadline tables were updated.

A1. How to use this document

This brief has two parts. **Part B (before this)** is your personal role: what you build, in what order, and how you know it is done. **Part A (this part)** is the single source of truth that all six roles share: the game design, every data value, every formula, every file name and every code interface.

- Part A is **identical in all six briefs**. If two people read Part A, they build parts that fit together without talking.
- If Part A and Part B ever disagree, **Part A wins**. Report the conflict to the Lead.
- Only the Lead may change Part A during the event. Every change is posted in the team Discord channel `#bible-changes` with a version number (v1.0 → v1.1 ...). Code and assets follow the latest posted version.
- All IDs in this document (`imager_ms`, `res_power`, `nile_watch` ...) are **exact strings**. Copy them character for character. File names, JSON keys and TypeScript identifiers must match exactly or integration breaks.
- The brief is written to be complete and self-contained: it can be handed to any collaborator or tool and executed without extra context. When you hand it over, include **both** Part B and Part A.

A2. Event facts and hard rules

Item	Value
Event	NASA Space Apps Cairo 2026 (local event of the global NASA Space Apps Challenge)
Venue	Elsewedy University of Technology (SUT), Cairo
Onsite registration	Friday 13 Nov 2026 from 9:00 AM (guidebook says 8:00 AM — arrive at 8:00)
Hackathon starts	**Friday 13 Nov, 1:00 PM** (call this H0)
Mentorship opens	Friday 2:00 PM, via Discord (plus onsite at peak hours)
Submission deadline	**Saturday 14 Nov, 1:00 PM** (H24). Late = rejected, no exceptions
Feature freeze	**Saturday 3:00 AM** (H14) — after this, only bug fixes, video, slides, submission
Our internal deadline	**Saturday 12:00 PM** — submission done, links verified
Closing ceremony	Saturday ~5:00 PM (People's Choice shortlist presents live)
Challenge	**Space Mission Design Game** (beginner / intermediate)
Project name	**DELTA-V TYCOON**
Team size	6 — max **5 onsite**; the 6th member works virtually on day 1 (may join onsite day 2)

Hard rules (breaking any of these can disqualify the team):

1. **No production work before H0 (Friday 1:00 PM).** Planning, splitting tasks, creating accounts, installing tools and creating an *empty* repository are allowed. Writing game code, drawing final art or recording before

H0 is not. Submitting a ready-made project is forbidden.

2. **Stick to the registered idea and challenge.** The idea cannot be changed after registration.
3. **Demo:** exactly **7 slides OR a 30-second video**, uploaded to a publicly accessible platform. It must not contain images, names or voices of anyone under 18.
4. **Language:** the solution must be in English. Any Arabic voice in the video needs English subtitles.
5. **Every submitted link must open without a password, sign-up or access request** (GitHub repo public, game link public, Drive files "anyone with the link").
6. **Submit twice:** first the local Cairo form, then the same material on the global NASA Space Apps site (needed for global judging). **Only the team leader submits.** No changes are allowed after submission.
7. **Work honestly:** credit every source and asset, respect copyright, and **disclose generative tools used** (the event code of ethics requires it). We list all tools in `README.md` → "Tools used".
8. **Do not use the NASA logo** ("meatball") or any agency logo in the game, video or slides — it implies endorsement.
9. Mentors can guide but will not write code.

Judging criteria (NASA Space Apps): Impact · Creativity · Validity (sound science, reliable data, feasible) · Relevance (answers the chosen challenge) · Presentation. Every feature below exists to score on at least one of these.

A3. The challenge, and how we answer every part of it

The challenge asks for an interactive game in which participants design, manage and simulate a complete space mission, make engineering decisions, manage limited resources, and see how each choice shapes mission success.

Official NASA resources and how the game uses each one (cite all four in README, `docs/SOURCES.md`, slides and the submission; this is our Validity and Relevance evidence):

Resource	Key idea	Where it lives in our game
R. Wessen, C. Borden, J. Ziemer, J. Kwok (JPL), *Space Mission Concept Development Using Concept Maturity Levels* (2013)	Concepts mature in levels: CML 1 Cocktail Napkin (science question, unique idea, launch timeframe, cost class) → CML 2 Initial Feasibility (key parameters, first calculations) → CML 3 Trade Space (explore how science, cost and risk change) → CML 4 Point Design (one design with acceptable margins). CML 2–4 loop back and forth. "Science partials" = how science return changes with one parameter. A checklist with green/yellow/red; a level is reached at ~80% green. Independent assessment beats self-assessment.	Briefing = CML 1; live meters = CML 2; Trade-Space Explorer = CML 3 (F-34); readiness review = CML 4 checklist with green/yellow/red and the 80% rule (F-35); the review is an independent assessor (Dr. Hassan's review board), not self-scored
J. Levri (NASA Small Spacecraft Systems Virtual Institute), *Design and Develop Science Missions*, Systems Engineering Webinar, 26 Mar 2025	A science mission = launch segment + space segment + ground segment + user segment. Process: Identify Need & Goals → Define Concept → Define Requirements → Design → Build & Verify → Implement. "Begin with the end in mind": start from what data users need. Every requirement has a verification method.	Phase stepper uses these six process names (A6). The four game roles map to the segments (A7). Each scenario names its user segment ("Who uses your data"). Requirements panel shows "Verified by" (F-36)
NASA Webb, *Build It Yourself* game (science.nasa.gov/mission/webb/game-build-it-yourself/)	Science requirements drive telescope design; infeasible options are greyed out; after launch the game shows a real mission with similar data; levels grow from small to large missions	Locked / "cannot reach" options with a reason; Mission Twin card after launch and in the report (F-33); difficulty levels from small (Cadet) to flagship (Flight Director)
NASA *Web-based Space Mission Visualization* (nasa.github.io/mission-viz/)	Browser orbit visualizations from Keplerian elements with an orbit propagator; Earth-to-Moon transfer animation	Orbit view draws Kepler ellipses and moves the spacecraft with Kepler's equation (faster near the planet); transfer animation for far orbits (B's brief)

Traceability matrix (challenge item → our feature ID, see A5). Judges check Relevance against exactly this list.

Challenge asks for	Our feature
Define a mission objective, then requirements	F-01 scenario briefing, F-02 requirements panel
Design a spacecraft (structure, power, batteries, comms, computer, attitude control, instruments)	F-03 spacecraft builder, F-04 instruments
Instruments differ in mass, power, cost, data and science value	F-04 (all five values shown on every card)
Choose orbit; explain coverage, observation, comms, lifetime	F-05 orbit picker with pros/cons + orbit view
Launch vehicles with capacity, cost, orbit capability; check mass vs vehicle	F-06 + check `launch_capacity`, `launch_destination`
Manage budget, mass, power, comms, data, time (lifetime)	F-07 live resource dashboard (7 meters incl. delta-v)
Every decision has a consequence; no single perfect solution	Formulas A9 (each choice moves 3+ meters); reference designs A17 show several winning designs
Clear UI, configuration, resource management	F-03..F-08
Single player and team of up to 4, roles, negotiation, collective decisions	F-18 online rooms, F-19 roles, F-20 Budget Council, F-21 sign-offs, F-22 role goals, F-23 same-screen mode
Simulation and evaluation	F-10 launch, F-11 operations sim, F-13 scoring
Feedback that explains *why*	F-08 check explanations, F-14 report "Why" section
Modify and re-simulate many times	F-15 retry keeping design, compare attempts
Difficulty levels, simple vs advanced components	F-16 Cadet / Engineer / Flight Director, basic/advanced tiers
Multiple scenarios	F-17 five scenarios + sandbox
Random events / unexpected constraints	F-12 ten events with decisions
Spacecraft visualization, real-time dashboards	F-09 composited spacecraft + orbit view, F-11 charts
Leaderboards, mission reports, save/share/compare	F-25, F-14, F-24
Competitive scenario (teams, same objective)	F-25 challenge codes (same seed for every team)
Sandbox mode	F-30
Use official NASA resources (A3)	F-33 Mission Twin, F-34 Trade-Space Explorer, F-35 Concept Maturity meter, F-36 verified requirements, NASA process stepper
Accessible to all knowledge levels; concepts taught in-game	F-26 Mission Academy, tooltips, F-27 tutorial

A4. Game concept

One-liner: *Delta-V Tycoon* puts you and up to three friends in charge of a small space agency's mission control. Pick a real-world-style mission, build the spacecraft part by part, pick an orbit and a rocket, argue over a shared budget, sign off together, launch — and watch every decision play out in a simulated mission, with a report that explains exactly why it succeeded or failed.

Why "Delta-V": delta-v (change in velocity, in m/s) is the "fuel budget" of space travel. In our game every orbit costs delta-v, the rocket equation turns delta-v into propellant mass, and propellant mass pushes you toward a bigger, more expensive rocket. This chain is the heart of real mission design and is visible on screen.

Why "Tycoon": you manage limited money like a business. Budget is split between the four roles, you can request money from teammates, and leftover reserve improves your score.

Core loop (one mission, 10–20 minutes):

1. **Briefing** — choose scenario + difficulty; read the need, the goals, who uses the data, and the requirements (F-01, F-02). *(NASA: Identify Need & Goals · CML 1 Cocktail Napkin)*
2. **Crew** — solo: pick your avatar; team: create/join a room, claim roles (F-18, F-19).
3. **Budget Council** — split the budget between the 4 roles; everyone agrees (F-20).
4. **Design** — each role configures its part; all 7 resource meters update live; the Trade-Space Explorer shows how science changes with any one parameter (F-03..F-07, F-09, F-34). *(NASA: Define Requirements & Design · CML 2–3)*
5. **Readiness review** — automatic GO/NO-GO checks with explanations, the Concept Maturity meter, and sign-offs from every role (F-08, F-21, F-35). *(NASA: Build & Verify · CML 4 Point Design)*
6. **Launch** — animated launch; can fail on real reliability numbers (F-10).
7. **Operations** — the mission plays out day by day at high speed; random events demand decisions from the role that owns them (F-11, F-12).
8. **Report** — score, stars, outcome, what went well, what hurt and why, what to try next, the real Mission Twin your design resembles, and who used your data (F-13, F-14, F-33).

9. **Iterate** — retry with the same design, change something, compare scores (F-15).

Game modes:

Mode	Description	Priority
Solo mission	One player owns all 4 roles	P0
Online team room	1–4 players on their own devices, room code + link, roles, council, sign-offs	P0-online (see cutoff rule A16)
Same-screen team	Same team mechanics on one device (fallback if online fails)	P0 (it is the solo code path)
Challenge code	Same scenario + difficulty + seed for every team → fair competition	P1
Sandbox	No budget limit, all components unlocked, events optional	P1
Agency career ("Tycoon")	Play scenarios in order; reputation raises next budget	P2 (only if everything else is done)

A5. Feature list (IDs, priority, owner)

Owners: **L** = Lead, **A** = Programmer A (core + rooms), **B** = Programmer B (UI + build), **C** = Character artist, **E** = Environment/asset artist, **Q** = QA tester.

ID	Feature	Pri	Owners
F-01	Scenario select + briefing (Dr. Hassan presents the mission)	P0	B, L (text), E (cards), C
F-02	Requirements panel (required instrument types, orbit, lifetime, budget, window)	P0	B, A
F-03	Spacecraft builder: bus, solar array (type + watts), RTG, battery (type + Wh), computer, attitude control, propulsion + delta-v margin	P0	A, B, E
F-04	Instrument catalog (cards show mass, power, cost, data/day, science, pointing need)	P0	A, B, E
F-05	Orbit picker with pros/cons and live orbit preview	P0	A, B, E
F-06	Launch vehicle picker with capacity per destination	P0	A, B, E
F-07	Resource dashboard: Mass, Power, Budget, Data, Comms, Time, Delta-v	P0	A, B, E
F-08	Readiness review: checks with severity, explanation, fix hint, concept link	P0	A, B, L
F-09	Spacecraft preview (composited pixel sprite) + orbit view canvas	P0	B, E
F-10	Launch sequence (animation, success or failure)	P0	B, E, A
F-11	Operations simulation with charts and speed control	P0	A, B
F-12	Random events with options, owner role, timer, suggestions	P0	A, B, L, E
F-13	Score (0–1000), stars, outcome class	P0	A, B
F-14	Mission report: breakdown, timeline, Why, Try next, print to PDF	P0	A, B, L
F-15	Retry with same design; attempt history and comparison	P1	B
F-16	Difficulty levels and component tiers	P0	A, B, L
F-17	Scenarios: nile_watch, storm_watch, ice_hunter (P0); red_relay, first_light (P1); sandbox (P1)	P0/P1	L, A
F-18	Online rooms: create/join by code or link, presence, max 4	P0-online	A, B
F-19	Roles: claim/release, role tabs, owner badge	P0	A, B, C
F-20	Budget Council + funds requests between roles	P0-online	A, B
F-21	Sign-offs (reset on any design change); event ownership; suggestions	P0	A, B
F-22	Role goals (personal objectives) feeding the team score	P0	A, B, L
F-23	Same-screen team mode (LocalRoom)	P0	A, B
F-24	Save designs, share codes/links, compare two designs	P1	A, B
F-25	Leaderboard (local P1, online P2) + challenge codes (P1)	P1/P2	A, B
F-26	Mission Academy glossary + "?" tooltips everywhere	P1	L, B
F-27	Guided tutorial on first Cadet mission	P1	L, B
F-28	Diverse crew, avatar select, portraits that react (neutral/happy/worried)	P0	C, B, L
F-29	Settings: volume, mute, text size, reduced motion	P1	B
F-30	Sandbox mode	P1	A, B
F-31	Agency career (Tycoon)	P2	A, B, L
F-32	In-room chat (short messages)	P1	A, B
F-33	Mission Twin: the real mission closest to your design (after launch + in report)	P1	L (data), A, B
F-34	Trade-Space Explorer: science, cost and margins vs one chosen parameter	P1	A, B
F-35	Concept Maturity meter (CML 2/3/4, green/yellow/red, 80% rule) in the review	P0	A, B

ID	Feature	Pri	Owners
F-36	Requirements panel with "Verified by" column; user segment line	P0	B, L

A6. Screens, routes and flow

The app is a single-page web app with a hash router (works on GitHub Pages without server config). Logical canvas is 16:9; target screens 1280×720 up to 1920×1080; must work at 1366×768.

Route	Screen	Notes
`#/`	Title	Play Solo · Create Team Room · Join Room · Mission Academy · Saved & Compare · Leaderboard · Settings · Credits
`#/solo`	Solo setup	Scenario cards, difficulty, avatar, optional challenge code → Start
`#/room/new`	Create room	Your name + avatar → creates code → goes to lobby
`#/join` and `#/join/:code`	Join room	Enter code (or prefilled), name, avatar
`#/room/:code`	Room shell	Renders the current phase: lobby → council → design → launch → ops → report
`#/play`	Solo shell	Same phase screens, driven by LocalRoom
`#/academy`	Mission Academy	Glossary cards (F-26)
`#/saved`	Saved designs + compare	(F-24)
`#/design/:code`	Open shared design	Decodes and opens it in solo shell
`#/leaderboard`	Leaderboard	Filter by scenario/difficulty/challenge code
`#/settings`, `#/credits`	Settings, credits	Credits list every asset + tool

Phase flow (both solo and team): `lobby` → `council` → `design` (review is a panel inside design) → `launch` → `ops` → `report`. In solo, `lobby` is skipped and `council` is a single confirm screen.

Phase stepper labels (NASA process, from Levri 2025) shown in the top bar: **Need & Goals** (briefing) → **Concept** (council) → **Requirements & Design** (design) → **Build & Verify** (review + launch) → **Implement** (ops) → **Results** (report). A small CML badge sits under Need & Goals (CML 1), Design (CML 2–3) and Build & Verify (CML 4, or the level actually reached).

A7. Game roles and team mechanics

Four **game roles** (separate from the six team-member jobs). Every mission always has all four. A player may hold several roles (solo player holds all four).

Role ID	Title	Owns (design fields)	Owns (events)	Role goal (counts 15 pts each)
`planner`	Mission Commander	orbit, mission lifetime, launch vehicle, delta-v margin	budget_cut, debris_conjunction	Total cost ≤ 90% of budget AND launch on time (launchMonth ≤ window)
`engineer`	Spacecraft Engineer	bus, solar array (+W), RTGs, battery (+Wh), computer, attitude control, propulsion	solar_storm, micrometeoroid, wheel_failure, battery_aging	Launch mass margin ≥ 10% AND power margin ≥ 15% AND battery covers eclipse
`science`	Science Lead	instruments	discovery, dust_storm	Science returned ≥ 100% of scenario target
`comms`	Communications & Data Officer	radio, high-gain antenna, ground network	ground_outage, computer_reset	≥ 95% of generated data returned to Earth

Mission segments (Levri 2025): Commander = launch segment (rocket, orbit, schedule) · Engineer and Science Lead = space segment (spacecraft and instruments) · Comms Officer = ground segment (radio, antennas, ground stations) · the **user segment** is the people who use the data, named on each scenario and in the report. The four roles cover every segment, so the team designs the *whole* mission, not just a spacecraft.

The goals pull against each other on purpose: Science wants more instruments; Engineer wants margins; Comms wants downlink headroom; Commander wants money left over. That is the negotiation the challenge asks for.

Budget Council (F-20): before design, the budget is split in percent between the four roles. Default split: planner 35, engineer 30, science 25, comms 10. Any player can drag the sliders (they always sum to 100); each player then presses **Agree**. Design opens when every connected player has agreed. Any slider change clears all agrees.

- Each role's **cost** (A9) is compared with its **allocation** = budget × split%. Going over allocation is allowed but shows a red "over allocation" badge on that role tab and a `warn` check.
- **Funds request:** a role can request an amount (in $M, step 0.5) from another role with a one-line reason. The receiving role's owner accepts or declines. Accepting moves that many percentage points of allocation. Requests appear as toasts and in a requests list.

Sign-offs (F-21): each role has a sign-off toggle in the review panel. **Any design change by anyone clears all four sign-offs** and shows a toast ("Science changed instruments — all sign-offs reset"). The host can press **LAUNCH** only when all four are signed. If any `fail` check exists, a confirmation dialog warns "NO-GO items present — launching will likely fail. Launch anyway?" Launching with failures is allowed on purpose: learning from failed missions is part of the challenge.

Event ownership: during operations each event belongs to one role. The player holding that role chooses an option; other players can tap an option to *suggest* it (suggestions appear as small avatars next to options). In team mode a timer runs (difficulty table); when it expires, the event's default option is applied.

A8. Game data (exact values)

All values are **simplified game values inspired by public data**, tuned for play. Units: mass kg, power W, money $M (millions of US dollars), data Gbit (gigabits), delta-v m/s, time months/years. The Lead turns these tables into JSON files in `src/data/` (field names in A14). Tier `basic` = always available; `advanced` = locked on Cadet difficulty.

A8.1 Orbits (`src/data/orbits.json`)

field	leo	sso	geo	lunar	l2	mars
name	Low Earth Orbit	Sun-Synchronous Orbit	Geostationary Orbit	Low Lunar Polar Orbit	Sun-Earth L2 Halo Orbit	Mars Polar Orbit
destination (rocket target)	LEO	SSO	GTO	TLI	L2	TMI
altitude km	500	700	35786	100	1500000	300
period min	95	99	1436	118	259200	113
sunFraction	0.62	0.65	0.99	0.60	1.0	0.65
eclipseMin	36	35	72	47	0	40
solarFactor	1.0	1.0	1.0	1.0	1.0	0.43
insertionDvMs	0	0	1500	900	50	1100
stationKeepingDvMsPerYear	25	10	50	100	3	15
disposalDvMs	60	150	11	0	1	20
naturalLifetimeYears (no propulsion)	4	25	999	0.5	0.1	50
commsFactor	1.0	1.0	0.05	0.002	0.0003	0.00002
lightDelaySec	0	0	0.12	1.3	5	750
radiation	1.0	1.2	2.0	2.5	2.5	3.0
opsCostMPerYear	2	2	2	4	4	4
cruiseMonths	0	0	0.1	0.2	1	7
magnetorquersWork	yes	yes	no	no	no	no
isNearEarth	yes	yes	yes	no	no	no
debrisRule	yes	yes	no	no	no	no

Contact hours per day (`contactHours`, by ground network; 0 = cannot reach):

orbit	campus	commercial	deepspace
leo	0.5	2.0	1.0
sso	0.4	3.0	1.0
geo	24	24	24
lunar	0	0	8
l2	0	0	4
mars	0	0	6

Player-facing explanations (shown in the orbit picker; Lead may polish wording):

- **leo** — 500 km up, circles Earth every 95 minutes. Close = sharp images and cheap launches. But you only see a ground station for a few minutes per pass, you spend 36 min of every orbit in Earth's shadow, and thin air slowly drags you down, so you need fuel to stay up and to deorbit at the end (space-debris rule).
- **sso** — a polar orbit tilted so you pass over every place at the same local time: perfect lighting for comparing images over months. Passes over polar ground stations often. Needs fuel to deorbit at end of life.
- **geo** — 35,786 km up; one orbit takes exactly one day, so you hover over the same spot — continuous weather watching and 24-hour contact with one station. Far away: blurry for small details, weaker radio signal, the rocket drops you in a transfer orbit and your spacecraft must burn 1,500 m/s to circularize.
- **lunar** — 100 km over the Moon's poles: maps the whole surface, including dark craters near the poles. Needs a deep-space antenna network; the Moon's lumpy gravity disturbs the orbit, so station-keeping fuel is high; no magnetic field, so magnetorquers do not work.
- **l2** — 1.5 million km from Earth on the side away from the Sun. The Sun, Earth and Moon are always behind you, so a telescope stays cold and steady with no eclipses. Far: deep-space network required, limited contact hours. The orbit is unstable, so small corrections are needed.
- **mars** — sunlight at Mars is only 43% as strong, so solar arrays must be larger. Radio signals take minutes to arrive (average 12.5 min one way), so the spacecraft must handle emergencies on its own. Getting there takes ~7 months and Mars launch windows open only every 26 months.

A8.2 Ground networks (`groundNetworks` in `src/data/components.json`)

id	name	gain	cost $M/yr	description
campus	University Ground Station (Cairo, 3 m dish)	1	0.5	One small dish. Cheap, few minutes per pass, cannot reach beyond Earth orbit.
commercial	Commercial Polar Network (several 5–7 m dishes)	3	3	Stations near the poles see polar orbits on almost every pass.
deepspace	Deep-Space Network (34–70 m dishes on 3 continents)	30	8	Huge dishes that can hear spacecraft at the Moon, L2 and Mars. Time is shared, so you get scheduled hours.

A8.3 Launch vehicles (`src/data/launchVehicles.json`)

Capacity in kg delivered to each destination. Missing destination = cannot go there.

id	name	class note	LEO	SSO	GTO	TLI	L2	TMI	cost $M	reliability	tier
spark	Spark	small launcher	300	200	–	–	–	–	8	0.93	basic
rideshare	Rideshare Slot	shared ride on a medium rocket, SSO only, max 300 kg	–	300	–	–	–	–	0.0065 × kg (min 0.5)	0.97	basic
arrow	Arrow	light-medium launcher	2300	1500	900	700	650	400	40	0.94	basic
workhorse	Workhorse	reusable medium-lift	17500	13000	5500	3500	3000	2500	70	0.98	basic
titan	Titan	heavy-lift	60000	45000	22000	15000	13000	12000	150	0.96	advanced

Rideshare: `costPerKgM` = 0.0065, `minCostM` = 0.5, `scheduleIntervalMonths` = 4 (launches only in months that are multiples of 4 — you may wait).

A8.4 Spacecraft components (`src/data/components.json`)

Buses (structure, thermal, wiring):

id	name	mass	cost	base load W	instrument slots	max total mass	spare wheel	tier
bus_cube	CubeSat 12U Bus	6	1	8	1	25	no	basic
bus_small	Small Bus	60	8	40	2	300	no	basic
bus_medium	Medium Bus	250	25	120	4	2000	yes	basic
bus_large	Large Bus	800	70	300	6	8000	yes	advanced

Solar arrays (player picks type and rated watts, slider 0–6000 W, step 50; `none` = no array):

id	name	W per kg	cost $M per W	degradation per year	tier
sa_si	Silicon Solar Array	25	0.001	0.03	basic
sa_gaas	Triple-Junction GaAs Array	60	0.002	0.015	advanced

RTG (count 0–3): `rtg` Radioisotope Power Unit — 45 kg, 110 W each, $60M each, decays 4%/yr, works in shadow and dust, tier advanced.

Batteries (type + capacity slider 50–5000 Wh, step 50):

id	name	Wh per kg	cost $M per Wh	usable depth of discharge	tier
bat_liion	Li-ion Battery	150	0.0005	0.4	basic
bat_liion_adv	High-Energy Li-ion Battery	250	0.001	0.5	advanced

Computers:

id	name	mass	power	cost	storage Gbit	compression	autonomy	rad-hard	tier
obc_basic	Standard Flight Computer	1.5	7	0.5	512	1.0	no	no	basic
obc_adv	Rad-Hard Computer with Compression	5	20	3	4096	0.5	yes	yes	advanced

Compression 0.5 halves the data volume without losing science. Autonomy lets the spacecraft react to emergencies on its own (needed when light delay > 60 s).

Attitude control (ADCS):

id	name	mass	power	cost	pointing accuracy (deg)	near-Earth only	tier
adcs_mag	Magnetorquers + Sun Sensors	2	3	0.3	5	yes	basic
adcs_wheels	Reaction Wheels + Star Tracker	15	40	4	0.05	no	basic
adcs_precision	Precision Pointing System	35	70	12	0.001	no	advanced

Propulsion (propellant is computed automatically from delta-v — see A9):

id	name	dry mass	Isp s	cost	power W	max propellant kg	electric	tier
prop_none	No Propulsion	0	0	0	0	–	no	basic
prop_coldgas	Cold-Gas Thrusters	5	65	0.5	0	20	no	basic
prop_mono	Hydrazine Monopropellant	20	220	3	10	–	no	basic
prop_biprop	Bipropellant Engine	60	315	8	20	–	no	basic
prop_electric	Electric Hall Thrusters	40	1600	12	10	–	yes	advanced

Electric propulsion needs `solarArrayW × solarFactor ≥ 1000` and adds `insertionDv / 250` months of transfer time.

Radios:

id	name	mass	power W (while transmitting)	cost	base rate Mbps	max rate Mbps	near-Earth only	rain fade	tier
radio_uhf	UHF Radio	1	10	0.2	0.0096	0.0096	yes	no	basic
radio_sband	S-Band Transmitter	3	20	1	2	10	no	no	basic
radio_xband	X-Band Transmitter	6	45	3	150	800	no	no	basic
radio_kaband	Ka-Band Transmitter	8	70	6	600	2000	no	yes	advanced

High-gain antenna (toggle): `hga` High-Gain Dish Antenna (1.5 m) — 20 kg, 0 W, $6M, gain ×20, works only if ADCS pointing ≤ 0.1°, tier basic.

A8.5 Instruments (`src/data/instruments.json`)

id	name	mass	power	cost	data Gbit/day	pointing need deg	tags	base science /yr	dev months	tier
cam_wide	Wide-Field Camera	5	10	2	20	1.0	optical	10	6	basic
cam_hires	High-Resolution Telescope Camera	40	60	25	200	0.05	optical_hr, optical	30	18	advanced
imager_ms	Multispectral Imager	25	40	15	80	0.1	multispectral	25	12	basic
imager_tir	Thermal Infrared Imager	20	50	12	40	0.1	thermal_ir	20	12	basic
radar_sar	Synthetic Aperture Radar	150	400	60	400	0.1	radar	40	24	advanced
sounder_atm	Atmospheric Sounder	60	120	30	30	0.5	sounder	30	18	basic
lightning_map	Lightning Mapper	40	70	20	50	0.2	lightning	20	15	basic
spec_neutron	Neutron Spectrometer	12	10	8	0.5	5.0	neutron	25	12	basic
lidar_alt	Laser Altimeter	15	35	10	5	0.05	altimeter	20	12	basic
magnetometer	Magnetometer	3	4	1.5	0.1	10	magnetic	10	6	basic
spec_atm	Atmospheric Spectrometer	20	30	14	15	0.1	spectrometer	25	15	basic

id	name	mass	power	cost	data Gbit/day	pointing need deg	tags	base science /yr	dev months	tier
det_particle	Radiation & Particle Detector	4	5	2	0.2	20	particles	8	6	basic
relay_uhf	Rover Relay Radio	30	100	15	2	5	relay	30	12	basic
tel_ir	Cryogenic Infrared Telescope	400	300	250	60	0.001	telescope_ir	80	30	advanced

`det_particle` has `special: "particle_warning"` (halves solar-storm damage risk). Plain-language "what it measures" text for each instrument is in the Lead's brief and goes into the JSON `whatItMeasures` field.

A8.6 Scenarios (`src/data/scenarios.json`)

id	name	default difficulty	required tag groups (all groups needed; any tag inside a group)	orbit fit	min life yrs	budget $M	launch window months	hard window	science target	events
nile_watch	Eyes on the Nile	cadet	[multispectral, optical, optical_hr]	sso 1.0, leo 0.8	2	120	36	no	110	3
storm_watch	Storm Watcher	engineer	[sounder, thermal_ir]	geo 1.0, sso 0.4, leo 0.3	5	320	36	no	300	4
ice_hunter	Ice Hunter	engineer	[neutron] + [altimeter, thermal_ir]	lunar 1.0	1	250	30	no	80	4
red_relay	Red Relay	director	[spectrometer] + [relay]	mars 1.0	2	300	26	yes	110	5
first_light	First Light	director	[telescope_ir]	l2 1.0, geo 0.5, leo 0.3	5	650	48	no	360	4
sandbox	Sandbox	engineer	none	all orbits 1.0	1	2000	120	no	100	3

Tag weights (science multiplier per instrument = highest weight among its tags; tags not listed use `defaultTagWeight` = 0.1):

- nile_watch: multispectral 1.0, radar 0.9, thermal_ir 0.8, optical_hr 0.7, optical 0.5
- storm_watch: sounder 1.0, thermal_ir 0.9, lightning 0.8, optical 0.4, particles 0.3
- ice_hunter: neutron 1.0, altimeter 0.9, thermal_ir 0.8, optical_hr 0.7, optical 0.6, particles 0.4, magnetic 0.3
- red_relay: spectrometer 1.0, relay 1.0, thermal_ir 0.7, optical_hr 0.6, optical 0.5, particles 0.4, magnetic 0.4
- first_light: telescope_ir 1.0, spectrometer 0.3, particles 0.2
- sandbox: defaultTagWeight 1.0

Event pools:

- nile_watch: solar_storm, ground_outage, debris_conjunction, discovery, micrometeoroid
- storm_watch: solar_storm, wheel_failure, budget_cut, computer_reset, discovery, battery_aging
- ice_hunter: solar_storm, wheel_failure, computer_reset, discovery, budget_cut, battery_aging
- red_relay: dust_storm, solar_storm, computer_reset, wheel_failure, discovery, budget_cut
- first_light: solar_storm, micrometeoroid, wheel_failure, computer_reset, discovery, budget_cut
- sandbox: all ten

Recommended orbit: nile_watch sso, storm_watch geo, ice_hunter lunar, red_relay mars, first_light l2, sandbox leo.

User segment ("Who uses your data", field `userSegment`): nile_watch: farmers, irrigation engineers and water ministries across the Nile Delta · storm_watch: weather forecasters, civil-protection agencies and airlines · ice_hunter: mission planners choosing safe, water-rich landing sites for future crews · red_relay: planetary scientists and the rover teams whose data you relay · first_light: astronomers studying the first galaxies and the atmospheres of distant planets · sandbox: you.

Mission Twins (field `twins`, F-33). Real missions to compare with. Matching: for each twin in the scenario's list, score = 2 if `orbit` equals the design's orbit, + 1 per tag shared between `keyTags` and the design's instrument tags; the highest score wins (ties: list order). Sandbox uses every twin from all scenarios.

scenario	name	agency	launched	orbit	keyTags	fact (shown in the card)
nile_watch	Landsat 9	NASA / USGS	2021	sso	multispectral, thermal_ir	Images the whole Earth in visible, infrared and thermal bands; used worldwide to track crops and water.

scenario	name	agency	launched	orbit	keyTags	fact (shown in the card)
nile_watch	Sentinel-2	ESA	2015	sso	multispectral	A pair of satellites revisiting every 5 days with 13 colour bands for vegetation monitoring.
storm_watch	GOES-16	NASA / NOAA	2016	geo	thermal_ir, lightning	Watches the Americas from geostationary orbit; carried the first lightning mapper in GEO.
storm_watch	Meteosat Third Generation Imager-1	ESA / EUMETSAT	2022	geo	thermal_ir, lightning	Watches Europe and Africa from GEO with an imager and a lightning imager.
ice_hunter	Lunar Prospector	NASA	1998	lunar	neutron	Its neutron spectrometer found signs of hydrogen, a hint of ice, at the lunar poles.
ice_hunter	Lunar Reconnaissance Orbiter	NASA	2009	lunar	altimeter, thermal_ir, neutron	Maps the Moon with a laser altimeter, a thermal radiometer and a neutron detector.
red_relay	Mars Reconnaissance Orbiter	NASA	2005	mars	relay, optical_hr, spectrometer	Studies Mars with a high-resolution camera and spectrometers, and relays rover data to Earth.
red_relay	ExoMars Trace Gas Orbiter	ESA / Roscosmos	2016	mars	spectrometer, relay	Hunts for methane in the Martian air and relays data for surface missions.
red_relay	MAVEN	NASA	2013	mars	spectrometer, particles, magnetic, relay	Studies how Mars lost its atmosphere, and also relays rover data.
first_light	James Webb Space Telescope	NASA / ESA / CSA	2021	l2	telescope_ir	A 6.5 m gold-coated infrared telescope at L2, shaded by a tennis-court-size sunshield.
first_light	Herschel Space Observatory	ESA	2009	l2	telescope_ir	A far-infrared telescope at L2 that studied how stars and galaxies form.

Story (one-line objective, shown on the scenario card):

- **Eyes on the Nile** — Monitor crops and water across the Nile Delta so farmers can save water and spot crop stress early.
- **Storm Watcher** — Watch dust storms and flash floods over Africa and the Middle East around the clock.
- **Ice Hunter** — Map water ice in the Moon's permanently shadowed craters to choose landing sites for future crews.
- **Red Relay** — Study methane and dust in the Martian atmosphere while relaying data home for rovers on the surface.
- **First Light** — Fly an infrared telescope to see the first galaxies and the atmospheres of distant planets.

A8.7 Difficulties (`src/data/difficulties.json`)

id	name	budget mult	event mult	launch failures	advanced unlocked	hints	decision timer s
cadet	Cadet	1.2	0.5	no	no	yes	60
engineer	Engineer	1.0	1.0	yes	yes	no	40
director	Flight Director	0.9	1.5	yes	yes	no	25

Number of events = `max(1, round(scenario.eventCount × eventMult))`. Hints = "suggested" badges on recommended choices plus Dr. Hassan tips.

A8.8 Random events (`src/data/events.json`)

Effect types (implemented by Programmer A): `loseScienceDays(days)` instruments off; `scienceMult(value, days?)` multiply science (no days = permanent); `powerMult(value, days?)`; `contactLossDays(days)` no downlink; `costM(value)` adds operations cost; `deltaVUse(value)` uses propellant (m/s); `scienceBonusPct(value)` adds value% of one year's planned science; `pointingMult(value)` multiplies ADCS pointing error permanently; `lifetimeYearsDelta(value)`; `dataMult(value, days)` multiplies data generated; `endMission(reason)`; `downgradeGround()` switches ground network one step cheaper; `immunity(eventId)` that event cannot happen again.

id	title	owner	conditions	options (✓ = safe/good choice)	default
solar_storm	Solar Storm!	engineer	any	**safe_mode** ✓ loseScienceDays 3; requires autonomy if light delay > 60 s · **keep_observing** risk 0.25 → scienceMult 0.7 permanent ("an instrument was damaged"); risk ×0.5 if `det_particle` carried, ×0.5 if `obc_adv`	safe_mode
micrometeoroid	Micrometeoroid Strike	engineer	any	auto: powerMult 0.9 permanent · **reduce_load** ✓ scienceMult 0.85 permanent · **full_ops** risk (0.6 if power margin after hit < 0, else 0.05) → loseScienceDays 10	reduce_load
ground_outage	Ground Station Outage	comms	any (twice as likely if `radio_kaband`)	**rent_backup** ✓ costM 1.5 · **wait** contactLossDays 3	wait
wheel_failure	Reaction Wheel Failure	engineer	ADCS is adcs_wheels or adcs_precision	**spare_wheel** ✓ nothing lost; requires bus with spare wheel · **magnetorquer_mode** pointingMult 5; requires near-Earth orbit · **accept** pointingMult 20	accept
budget_cut	Budget Cut!	planner	any	**pay_reserve** ✓ costM = 10% of budget; requires reserve ≥ that amount · **shorten** lifetimeYearsDelta −1 · **cheaper_ground** downgradeGround	shorten
computer_reset	Computer Reset	comms	any (auto-handled with `obc_adv`: info only, loseScienceDays 0)	**reload** loseScienceDays 1 + contactLossDays 1 · **patch** ✓ costM 0.5 + loseScienceDays 2 + immunity computer_reset	reload
debris_conjunction	Space Debris Close Approach	planner	orbits leo, sso, geo	**avoid** ✓ deltaVUse 2 + loseScienceDays 1; requires propulsion · **stay** risk 0.02 → endMission "collision with debris"	avoid if possible, else stay
discovery	Unexpected Discovery!	science	any	**follow_up** ✓ scienceBonusPct 15 + dataMult 3 for 5 days · **stick_to_plan** nothing	stick_to_plan
dust_storm	Global Dust Storm on Mars	science	orbit mars	**observe** ✓ scienceBonusPct 25 + relay science ×0.5 for 30 days · **relay_focus** scienceBonusPct 5	relay_focus
battery_aging	Battery Wearing Out	engineer	any orbit with eclipse > 0	auto: battery capacity ×0.8 permanent · **eclipse_saving** ✓ scienceMult 0.9 permanent · **push_on** risk (0.3 if battery now below requirement, else 0.02) → loseScienceDays 15	eclipse_saving

Every option has an `explanation` sentence shown after choosing (written by the Lead): what happened and why, in plain words.

A9. Formulas (Programmer A implements exactly; QA verifies with A17)

Constants (`src/data/constants.json`): `G0 = 9.81`, `SOLAR_EFFICIENCY = 0.85`, `INTEGRATION_OVERHEAD = 0.20`, `TANK_FRACTION = 0.10`, `RADIO_STANDBY_W = 2`, `MIN_BATTERY_WH = 50`, `HGA_MAX_POINTING_DEG = 0.1`, `DELAY_COST_M_PER_MONTH = 1.5`, `BASE_DEV_MONTHS = 12`, `ADVANCED_DEV_MONTHS = 2`, `MARS_SYNODIC_MONTHS = 26`, `ELECTRIC_MIN_W = 1000`, `ELECTRIC_DV_PER_MONTH = 250`, `MAX_LIFETIME_YEARS = 10`, `DAYS_PER_YEAR = 365`.

Mass

```
dryKg = bus.massKg + Σ instrument.massKg
      + (solarArrayW / array.wPerKg)            [0 if no array]
      + rtgCount × rtg.massKg
      + batteryWh / battery.whPerKg
      + computer.massKg + adcs.massKg + propulsion.dryMassKg
      + radio.massKg + (hga ? hga.massKg : 0)

dvRequired   = orbit.insertionDvMs + orbit.stationKeepingDvMsPerYear × lifetimeYears + orbit.disposalDvMs
dvWithMargin = dvRequired × (1 + deltaVMarginPct / 100)
x            = exp(dvWithMargin / (propulsion.ispS × G0)) − 1        [x = 0 if prop_none]
propellantKg = dryKg × x / (1 − TANK_FRACTION × x)                 [if denominator ≤ 0.05 → infeasible]
if propulsion.maxPropellantKg and propellantKg > max: propellantKg = max (capability reduced)
tankKg   = TANK_FRACTION × propellantKg
totalKg  = dryKg + propellantKg + tankKg
dvCapability = ispS × G0 × ln((dryKg + tankKg + propellantKg) / (dryKg + tankKg))
launchCapacityKg = vehicle.capacityKg[orbit.destination]  (undefined → cannot reach)
launchMarginPct  = (launchCapacityKg − totalKg) / launchCapacityKg × 100
```

Communications and data

```
contactHours   = orbit.contactHours[groundNetworkId]
hgaEffective   = hga AND adcs.pointingDeg ≤ HGA_MAX_POINTING_DEG
rateMbps       = min(radio.maxRateMbps,
                     radio.baseRateMbps × orbit.commsFactor × ground.gain × (hgaEffective ? hga.gain : 1))
downlinkGbitPerDay  = rateMbps × contactHours × 3.6
generatedGbitPerDay = Σ instrument.dataGbitPerDay × computer.compression
returnPct = generated = 0 ? 100 : min(100, downlink / generated × 100)
```

Power

```
arrayAvgW   = solarArrayW × orbit.solarFactor × orbit.sunFraction × SOLAR_EFFICIENCY
rtgW        = rtgCount × rtg.powerW
generationW = arrayAvgW + rtgW
loadW = bus.baseLoadW + Σ instrument.powerW + computer.powerW + adcs.powerW + propulsion.powerW
      + radio.powerW × min(1, contactHours / 24) + RADIO_STANDBY_W
powerMarginPct    = (generationW − loadW) / loadW × 100
batteryRequiredWh = max(MIN_BATTERY_WH, max(0, loadW − rtgW) × orbit.eclipseMin / 60 / battery.depthOfDischarge)
batteryOk         = batteryWh ≥ batteryRequiredWh
```

Pointing quality per instrument: `quality = clamp(instrument.pointingReqDeg / adcs.pointingDeg, 0.1, 1)`.

Cost (all $M)

```
hwEngineer = bus.costM + solarArrayW × array.costMPerW + rtgCount × rtg.costM + batteryWh × battery.costMPerWh
           + computer.costM + adcs.costM + propulsion.costM
hwScience  = Σ instrument.costM
hwComms    = radio.costM + (hga ? hga.costM : 0)
launchCost = vehicle.costM, or for rideshare max(minCostM, costPerKgM × totalKg)
cost.engineer = hwEngineer × (1 + INTEGRATION_OVERHEAD)
cost.science  = hwScience  × (1 + INTEGRATION_OVERHEAD)
cost.comms    = hwComms    × (1 + INTEGRATION_OVERHEAD) + ground.costMPerYear × lifetimeYears
cost.planner  = launchCost + orbit.opsCostMPerYear × lifetimeYears + delayCostM
totalCost  = Σ cost.role
budgetM    = scenario.budgetM × difficulty.budgetMult
reservePct = (budgetM − totalCost) / budgetM × 100
allocationM[role] = budgetM × split[role] / 100
```

Time

```
advancedCount  = number of selected components/instruments with tier "advanced"
devMonths      = BASE_DEV_MONTHS + max(instrument.devMonths, 0) + ADVANCED_DEV_MONTHS × advancedCount
launchMonth    = devMonths, rounded UP to a multiple of 4 if vehicle is rideshare
if scenario.hardLaunchWindow:                        (Mars)
   window = launchWindowMonths
   while launchMonth > window: window += MARS_SYNODIC_MONTHS
   launchMonth = (window == launchWindowMonths) ? launchMonth : window   (wait for the next window)
delayMonths    = max(0, launchMonth − launchWindowMonths)
delayCostM     = DELAY_COST_M_PER_MONTH × delayMonths
transferMonths = orbit.cruiseMonths + (electric ? orbit.insertionDvMs / ELECTRIC_DV_PER_MONTH : 0)
```

Example: Mars window at month 26, development finishes at month 30 → launch at month 52 → delay 26 months → $39M extra and the Commander's role goal fails.

A10. Readiness checks (runChecks)

Each check returns `{ id, severity, role, title, message, fixHint, conceptId }`. Messages explain **why** in plain language with numbers inserted. Exact English text is written by the Lead in `src/data/checks.json` (keys = check id; `{placeholders}` filled by code).

id	role	fail when	warn when	concept
launch_destination	planner	vehicle has no capacity for orbit destination (or rideshare not SSO)	–	payload_capacity
launch_capacity	planner	totalKg > launchCapacityKg	margin < 10%	payload_capacity
bus_mass	engineer	totalKg > bus.maxMassKg	totalKg > 90% of max	mass_budget
bus_slots	science	instruments > bus.instrumentSlots	–	mass_budget
power_balance	engineer	generationW < loadW	margin < 15%	power_budget
battery_eclipse	engineer	batteryWh < 50% of required	batteryWh < required	eclipse
deltav_capability	engineer	dvCapability < orbit.insertionDvMs, or rocket equation infeasible	dvCapability < dvRequired (lifetime will be cut)	delta_v
electric_power	engineer	electric propulsion and solarArrayW × solarFactor < 1000	–	delta_v
adcs_environment	engineer	adcs.nearEarthOnly and orbit not near Earth	–	pointing
pointing_quality	science	–	any instrument quality < 1 (list them)	pointing
hga_pointing	comms	–	hga selected but not effective	antenna_gain
radio_range	comms	radio.nearEarthOnly and orbit not near Earth, or contactHours = 0 for chosen network	–	ground_station
data_return	comms	returnPct < 25	returnPct < 95	data_rate
storage	comms	–	daily excess (generated − downlink) × 3 > storage	data_rate
budget_total	planner	totalCost > budgetM	reservePct < 5	mission_cost
role_allocation	(each role)	–	cost.role > allocationM.role	mission_cost
launch_window	planner	–	launchMonth > window (show delay months and cost)	launch_window
required_instruments	science	any required tag group not satisfied	–	mission_requirements
orbit_fit	planner	orbit not in scenario.orbitFit	fit < 1	orbit
lifetime_min	planner	–	lifetimeYears < scenario.minLifetimeYears	mission_lifetime
debris_disposal	planner	–	orbit.debrisRule and (prop_none or dvCapability < dvRequired)	space_debris
natural_lifetime	planner	–	prop_none and lifetimeYears > orbit.naturalLifetimeYears	orbit

Sort: fail first, then warn, then pass; group by role in the UI.

Concept Maturity meter (F-35, from JPL's CML paper). Treat each check as a checklist item: pass = green, warn = yellow, fail = red. `greenPct = passCount / totalChecks × 100`. Level reached: **CML 4 "Point Design"** if greenPct ≥ 80 and no red; else **CML 3 "Trade Space"** if greenPct ≥ 50; else **CML 2 "Initial Feasibility"**. The review header shows: "Concept Maturity: CML 3 Trade Space · 72% green · 4 yellow · 1 red — reach 80% green with no red for CML 4 Point Design". Launching below CML 4 is allowed (see A7), but the NO-GO dialog names the level reached.

Template variables. `runChecks` passes **all** of these to **every** template, so any text may use any of them. Numbers have at most 1 decimal; money is pre-formatted like `$70.8M`; percentages are plain numbers (the template writes the `%`).

```
{totalKg} {capKg} {launchMargin} {vehicle} {destination} {busName} {busMaxKg} {slots}
{instrCount} {genW} {loadW} {powerMargin} {batteryWh} {batteryReqWh} {eclipseMin}
{dvRequired} {dvCapability} {insertionDv} {rateMbps} {downlink} {generated} {returnPct}
{storage} {totalCost} {budget} {reservePct} {role} {roleCost} {roleAlloc} {launchMonth}
{window} {delayMonths} {delayCost} {lifetime} {minLifetime} {orbit} {fit} {recommendedOrbit}
```

`{missingTags}` `{blurredList}` `{adcsName}` `{adcsDeg}` `{radio}` `{ground}` `{disposalDv}` `{naturalLife}` `{lightDelay}`

`{role}` is the role title (e.g. "Science Lead") of the check; `{missingTags}` lists missing groups as words ("neutron, or altimeter / thermal infrared"); `{blurredList}` lists instruments with quality < 1 as "Laser Altimeter (20%)"; `{fit}` is a percentage; `{lightDelay}` is formatted ("12.5 min").

A11. Mission simulation (`createSimulation`)

Deterministic: same inputs + same seed + same decisions = same result on every device. Random numbers come only from a seeded PRNG (mulberry32) created from `seed`.

Before day 0:

1. **Launch:** if `totalKg > launchCapacityKg` or destination unreachable → outcome `launch_failure` (reason: "too heavy for the rocket" / "rocket cannot reach this orbit"). Else if difficulty has launch failures and `rng() > vehicle.reliability` → `launch_failure` (reason: "launch vehicle anomaly"). Cadet never has random launch failure.
2. **Orbit insertion:** if `dvCapability < orbit.insertionDvMs` → outcome `orbit_failure`.
3. **Event schedule:** count = events per difficulty (A8.7). Filter the pool by conditions. Draw events without repetition (repeat allowed only if the pool is smaller than the count). Assign each a day = `floor(rng() × plannedDays × 0.9) + 1`, sorted ascending. Draw each event's risk roll in advance in schedule order, so every client gets the same rolls.
4. `plannedDays = round(lifetimeYears × 365)`. `endDay` = minimum of planned days, natural lifetime days (if prop_none), and the day propellant for station-keeping runs out (dvCapability − insertion − disposal reserve, spent at stationKeepingDvMsPerYear / 365 per day).

Each day d (1 … endDay):

```
years       = d / 365
arrayW_t    = solarArrayW × (1 − array.degradationPerYear)^years
rtgW_t      = rtgCount × rtg.powerW × (1 − rtg.decayPerYear)^years
generation  = arrayW_t × solarFactor × sunFraction × SOLAR_EFFICIENCY × powerMult_active + rtgW_t
nonInstrumentLoad = loadW − Σ instrument.powerW
if generation < nonInstrumentLoad → outcome "power_failure", stop (computer and radio cannot run)
duty        = clamp((generation − nonInstrumentLoad) / Σ instrument.powerW, 0, 1)
duty       *= batteryOk_t ? 1 : max(0.2, batteryWh_t / batteryRequiredWh)
duty        = 0 on days covered by loseScienceDays
generatedToday = generatedGbitPerDay × duty × dataMult_active
capacity    = downlinkGbitPerDay  (0 during contactLossDays)
queue       = stored + generatedToday;  sent = min(queue, capacity);  stored = queue − sent
if stored > computer.storageGbit: lost += stored − storage; stored = storage
scienceGeneratedToday = Σ_i baseScience_i × tagWeight_i × orbitFit × quality_i × duty × scienceMult_active / 365
   (relay instruments use relayMult during dust storm)
totals: dataGenerated += generatedToday; dataReturned += sent; scienceGenerated += scienceGeneratedToday
```

Science returned = `scienceGenerated × (dataReturned / dataGenerated)` (1 if no data generated) + bonuses from `scienceBonusPct` (value% × one year of planned science at full duty).

Event day: the stepper stops with `pendingEvent`. Auto-effects apply first. If the computer auto-handles it (computer_reset with obc_adv), there is no decision. Otherwise wait for `decide(optionId)`. Options whose `requires` is not met are shown disabled with the reason. Apply the option's effects; if it has a risk, use the pre-drawn roll for this event: `roll < chance × Π(reductions)` → apply risk effects and mark the event outcome **bad**. Log the event, the chosen option, who chose it, and the result text.

Samples for charts: record a sample every `max(1, floor(endDay / 200))` days: `{ day, generationW, loadW, duty, stored, dataGeneratedCum, dataReturnedCum, scienceCum, costCum }`.

Outcomes: `launch_failure`, `orbit_failure`, `power_failure`, `ended_early` (propellant, natural decay, collision or budget shortened below the minimum), `completed`.

A12. Scoring (`scoreMission`) — maximum 1000

Part	Max	Rule
Objectives	300	150 × (satisfied required groups / total groups; 1 if none) + 50 × orbitFit + 100 × min(1, achievedYears / minLifetimeYears)
Science	300	300 × min(1, scienceReturned / scienceTarget)

Part	Max	Rule
Efficiency	200	Budget 100 × clamp(finalReservePct / 15, 0, 1) (0 if over budget) · Mass margin 50: m<0 → 0; 0–5 → linear 0→50; 5–60 → 50; 60–95 → linear 50→30; ≥95 → 30 · Power margin 50: p<0 → 0; 0–10 → linear 0→50; 10–60 → 50; 60–150 → linear 50→25; ≥150 → 25
Operations	100	100 − 40 × dataLostFraction − 15 × (bad event outcomes) − 20 × (debris rule violated) − 30 × (power_failure); min 0
Teamwork	100	15 × each role goal met (A7) + 20 if no `fail` check at launch + 20 if the review was completed (solo: review panel opened and confirmed; team: all four sign-offs)

If outcome is `launch_failure` or `orbit_failure`, Objectives, Science and Operations are 0. Efficiency and Teamwork still count, so a good plan with bad luck still earns some points.

Stars: ★ ≥ 450 · ★★ ≥ 650 · ★★★ ≥ 850. **Outcome class:** `success` = all required groups satisfied AND achievedYears ≥ minimum AND scienceReturned ≥ 70% of target; `partial` = reached orbit and scienceReturned > 0; `failure` = otherwise. **Rank title:** under 450 "Trainee", 450+ "Mission Specialist", 650+ "Flight Engineer", 850+ "Flight Director".

Report explanations ("Why"): the core produces a list of `Insight { kind: 'good'|'bad', role, text, conceptId }` from rules. At least these rules: launch margin, power margin, battery, data return, lost data, pointing degraded, lifetime achieved vs planned, budget reserve, each bad event, each good event choice, missing required instrument, orbit fit < 1, debris rule. **Try next:** up to 3 concrete suggestions from the worst insights (e.g. "Your X-band link returned only 41% of the data. Try the commercial ground network or a high-gain antenna.").

A13. Online rooms (Firebase Realtime Database)

Why Firebase: the game stays a static site on GitHub Pages, and Firebase RTDB gives realtime sync, presence and anonymous sign-in with no server to run. The free Spark plan covers this event. Anonymous auth is invisible to players: no sign-up, which satisfies the "no registration" rule.

Room code: 5 characters from `ABCDEFGHJKMNPQRSTUVWXYZ23456789` (no 0/O/1/I/L). Join link: `https://<github-user>.github.io/delta-v-tycoon/#/join/ABCDE`. Max 4 connected players.

Database layout (`rooms/{code}`):

```
rooms/{code}/
  meta:      { code, hostUid, createdAt, scenarioId, difficulty, seed, phase, teamName, version: 1, opsSpeed?,
      opsPaused? }
  players/{uid}: { uid, name, characterId, connected, joinedAt, lastSeen }
  roles/{role}:  uid | null                    role = planner|engineer|science|comms
  council:   { split: { planner, engineer, science, comms }, agreed: { [uid]: true } }
  requests/{id}: { id, fromRole, toRole, amountM, reason, status: pending|accepted|declined, createdAt }
  design/planner:  { orbitId, lifetimeYears, launchVehicleId, deltaVMarginPct }
  design/engineer: { busId, solarArrayId, solarArrayW, rtgCount, batteryId, batteryWh, computerId, adcsId,
      propulsionId }
  design/science:  { instrumentIds: string[] }
  design/comms:    { radioId, hga, groundNetworkId }
  signoffs/{role}: boolean
  reviewDone: boolean
  decisions/{eventIndex}: { optionId, byUid, at }
  suggestions/{eventIndex}/{uid}: optionId
  chat/{id}: { uid, name, text (≤200 chars), at }
  result: { score, stars, outcome, designCode, at }
leaderboard/{scenarioId}/{id}: { teamName, difficulty, seed, score, stars, outcome, designCode, at }
```

Rules of play (enforced in the client; security rules in Programmer A's brief):

- The host creates the room, picks scenario and difficulty, and moves `phase` forward. Seed = random 32-bit integer at creation (or taken from a challenge code).
- Players claim roles with a transaction (first claim wins). Unclaimed roles can be claimed by anyone; one player may hold several roles. The host can release any role.
- Each role writes only its own `design/{role}` subtree. Every design write is a multi-path update that also sets all four `signoffs/*` to false.
- Operations: every client runs the same deterministic simulation locally. At each event the sim waits until `decisions/{eventIndex}` exists. The owner of the event's role writes it; when the timer expires, any client writes the default (first write wins).

- At the end, the host writes `result`. Optional (P2): post to `leaderboard`.
- Presence: `players/{uid}/connected` goes false through `onDisconnect`. If the host is gone > 30 s, the earliest-joined connected player takes over host (P1).

Same-screen and solo: `LocalRoom` implements the same `RoomApi` in memory. The single local player owns all roles. The UI is identical; only the adapter differs. This is also the fallback if the network fails.

Codes:

- **Design share code** = `lz-string.compressToEncodedURIComponent(JSON.stringify({ v:1, scenarioId, difficulty, seed, teamName, design, decisions }))`. Link `#/design/<code>`.
- **Challenge code** = `DVT-<scenarioId>-<difficulty>-<seed in base36>`, e.g. `DVT-storm_watch-engineer-k3x9q1`. Entering it fixes the scenario, difficulty and event seed for fair team-vs-team competition.

A14. Tech stack, repository and code contracts

Stack: Vite + React 18 + TypeScript (strict). State: Zustand. Network: Firebase JS SDK (modular, v10+: app, auth, database). Charts: Recharts. Share codes: lz-string. Tests: Vitest. Lint: ESLint + Prettier defaults. Deploy: GitHub Actions → GitHub Pages. Fonts (self-hosted via npm, SIL Open Font License): `@fontsource/press-start-2p` (titles), `@fontsource/vt323` (body text). Node 20 LTS.

Repository: `github.com/<lead-account>/delta-v-tycoon` — **public**. Vite `base: '/delta-v-tycoon/'`.

```
delta-v-tycoon/
  index.html  package.json  vite.config.ts  tsconfig.json  .eslintrc.cjs
  README.md            (L)   CREDITS.md (E + L)   LICENSE (MIT, L)
  database.rules.json  (A)
  .github/workflows/deploy.yml (B)
  docs/GDD.md (L)  docs/TEST_PLAN.md (Q)  docs/TEST_REPORT.md (Q)  docs/SOURCES.md (L)
  scripts/make-placeholders.mjs (B)   scripts/balance.ts (A)   scripts/check-assets.mjs (Q)
  public/assets/...    (C, E)  — exact paths in A15
  src/
    main.tsx  App.tsx  router.tsx         (B)
    assets.ts                             (B)  id → path map
    data/*.json                           (L)  content
    data/index.ts                         (A)  typed loader + validation
    core/types.ts rng.ts catalog.ts budgets.ts checks.ts sim.ts events.ts score.ts insights.ts share.ts index.ts
      (A)
    core/__tests__/*.test.ts              (A)
    net/roomApi.ts localRoom.ts firebaseRoom.ts firebase.ts roomCodes.ts leaderboard.ts   (A)
    state/store.ts                        (B)
    ui/screens/*.tsx ui/components/*.tsx ui/styles/*.css   (B)
    scenes/SpacecraftPreview.tsx OrbitView.tsx LaunchScene.tsx   (B)
    audio/sound.ts                        (B)
```

Folder ownership = merge safety. Only the owner edits a folder. If you need a change in someone else's folder, ask them on Discord. Everyone commits small and often: `git pull --rebase` → commit → push. Artists may upload through the GitHub web UI into `public/assets/...`. Commit messages: `[A] sim: add events`, `[C] portraits: nour`, etc.

A14.1 `src/core/types.ts` — the contract (Programmer A writes this file first, by H1:30)

```
export type Role = 'planner' | 'engineer' | 'science' | 'comms';
export const ROLES: Role[] = ['planner', 'engineer', 'science', 'comms'];
export type OrbitId = 'leo' | 'sso' | 'geo' | 'lunar' | 'l2' | 'mars';
export type Destination = 'LEO' | 'SSO' | 'GTO' | 'TLI' | 'L2' | 'TMI';
export type GroundNetworkId = 'campus' | 'commercial' | 'deepspace';
export type DifficultyId = 'cadet' | 'engineer' | 'director';
export type ScenarioId = 'nile_watch' | 'storm_watch' | 'ice_hunter' | 'red_relay' | 'first_light' | 'sandbox';
export type Tier = 'basic' | 'advanced';
export type Severity = 'pass' | 'warn' | 'fail';
export type Phase = 'lobby' | 'council' | 'design' | 'launch' | 'ops' | 'report';
export type Outcome = 'launch_failure' | 'orbit_failure' | 'power_failure' | 'ended_early' | 'completed';
export type OutcomeClass = 'success' | 'partial' | 'failure';
```

```
interface Item { id: string; name: string; tier: Tier; description: string }

export interface Orbit {
  id: OrbitId; name: string; destination: Destination; altitudeKm: number; periodMin: number;
  sunFraction: number; eclipseMin: number; solarFactor: number;
  insertionDvMs: number; stationKeepingDvMsPerYear: number; disposalDvMs: number;
  naturalLifetimeYears: number; commsFactor: number; contactHours: Record<GroundNetworkId, number>;
  lightDelaySec: number; radiation: number; opsCostMPerYear: number; cruiseMonths: number;
  magnetorquersWork: boolean; isNearEarth: boolean; debrisRule: boolean;
  description: string; pros: string[]; cons: string[];
}
export interface LaunchVehicle extends Item {
  classNote: string; capacityKg: Partial<Record<Destination, number>>; costM: number;
  costPerKgM?: number; minCostM?: number; reliability: number; scheduleIntervalMonths?: number;
}
export interface Bus extends Item { massKg: number; costM: number; baseLoadW: number; instrumentSlots: number;
      maxMassKg: number; hasSpareWheel: boolean }
export interface SolarArray extends Item { wPerKg: number; costMPerW: number; degradationPerYear: number }
export interface Rtg extends Item { massKg: number; powerW: number; costM: number; decayPerYear: number }
export interface Battery extends Item { whPerKg: number; costMPerWh: number; depthOfDischarge: number }
export interface Computer extends Item { massKg: number; powerW: number; costM: number; storageGbit: number;
      compression: number; autonomy: boolean; radHard: boolean }
export interface Adcs extends Item { massKg: number; powerW: number; costM: number; pointingDeg: number;
      nearEarthOnly: boolean }
export interface Propulsion extends Item { dryMassKg: number; ispS: number; costM: number; powerW: number;
      maxPropellantKg?: number; electric: boolean }
export interface Radio extends Item { massKg: number; powerW: number; costM: number; baseRateMbps: number;
      maxRateMbps: number; nearEarthOnly: boolean; rainFade: boolean }
export interface Hga extends Item { massKg: number; costM: number; gain: number; maxPointingDeg: number }
export interface GroundNetwork { id: GroundNetworkId; name: string; gain: number; costMPerYear: number;
      description: string }
export interface Instrument extends Item {
  massKg: number; powerW: number; costM: number; dataGbitPerDay: number; pointingReqDeg: number;
  tags: string[]; baseScience: number; devMonths: number; whatItMeasures: string; special?: 'particle_warning';
}
export interface Components {
  buses: Bus[]; solarArrays: SolarArray[]; rtg: Rtg; batteries: Battery[]; computers: Computer[];
  adcs: Adcs[]; propulsion: Propulsion[]; radios: Radio[]; hga: Hga; groundNetworks: GroundNetwork[];
}
export interface Scenario {
  id: ScenarioId; name: string; tagline: string; defaultDifficulty: DifficultyId; briefing: string;
  requiredTagGroups: string[][]; orbitFit: Partial<Record<OrbitId, number>>; minLifetimeYears: number;
  budgetM: number; launchWindowMonths: number; hardLaunchWindow: boolean; scienceTarget: number;
  tagWeights: Record<string, number>; defaultTagWeight: number; eventPool: string[]; eventCount: number;
  recommendedOrbit: OrbitId; learningGoals: string[]; userSegment: string; twins: MissionTwin[];
}
export interface MissionTwin { name: string; agency: string; launchYear: number; orbit: OrbitId; keyTags:
      string[]; fact: string }
export interface Difficulty { id: DifficultyId; name: string; budgetMult: number; eventMult: number;
      launchFailures: boolean; advancedUnlocked: boolean; hints: boolean; decisionTimerSec: number; description:
      string }

export type EffectType = 'loseScienceDays' | 'scienceMult' | 'powerMult' | 'contactLossDays' | 'costM' |
      'deltaVUse'
  | 'scienceBonusPct' | 'pointingMult' | 'lifetimeYearsDelta' | 'dataMult' | 'endMission' | 'downgradeGround'
  | 'immunity' | 'batteryMult' | 'relayMult';
export interface EventEffect { type: EffectType; value?: number; days?: number; reason?: string; eventId?: string
      }
export interface EventRisk {
  chance: number;                      // base chance 0..1
  chanceRule?: 'power_margin_after_hit' | 'battery_below_required';  // dynamic chance (A8.8)
  chanceIfRule?: number;               // chance used when the rule is true
  effects: EventEffect[];
  reductions?: { ifInstrument?: string; ifComputer?: string; mult: number }[];
}
export interface EventOption {
  id: string; label: string; description: string; effects: EventEffect[]; risk?: EventRisk; good: boolean;
  requires?: { propulsion?: boolean; nearEarth?: boolean; busSpareWheel?: boolean; reservePctOfBudget?: number;
      autonomyIfDelayOverSec?: number };
  explanation: string; badExplanation?: string;
}
export interface GameEvent {
  id: string; title: string; ownerRole: Role; description: string; conceptId: string;
  conditions?: { orbits?: OrbitId[]; adcsIn?: string[]; needsEclipse?: boolean };
  weightIfRadio?: { radioId: string; weight: number };
  autoEffects?: EventEffect[]; autoHandledByComputer?: string; autoHandledText?: string;
  options: EventOption[]; defaultOptionId: string; defaultOptionIdIfNoPropulsion?: string;
}

export interface DesignPlanner { orbitId: OrbitId; lifetimeYears: number; launchVehicleId: string;
      deltaVMarginPct: number }
export interface DesignEngineer { busId: string; solarArrayId: string | 'none'; solarArrayW: number; rtgCount:
      number; batteryId: string; batteryWh: number; computerId: string; adcsId: string; propulsionId: string }
```

```
export interface DesignScience { instrumentIds: string[] }
export interface DesignComms { radioId: string; hga: boolean; groundNetworkId: GroundNetworkId }
export interface Design { planner: DesignPlanner; engineer: DesignEngineer; science: DesignScience; comms:
      DesignComms }
export type BudgetSplit = Record<Role, number>;   // percents, sum 100

export interface Line { label: string; value: number; role: Role }
export interface Budgets {
  mass: { dryKg: number; propellantKg: number; tankKg: number; totalKg: number; busMaxKg: number;
      launchCapacityKg: number | null; launchMarginPct: number | null; lines: Line[] };
  power: { generationW: number; loadW: number; marginPct: number; rtgW: number; batteryWh: number;
      batteryRequiredWh: number; batteryOk: boolean; eclipseMin: number; lines: Line[] };
  data: { generatedGbitPerDay: number; downlinkGbitPerDay: number; rateMbps: number; contactHoursPerDay: number;
      returnPct: number; storageGbit: number; hgaEffective: boolean };
  deltaV: { requiredMs: number; withMarginMs: number; capabilityMs: number; ispS: number; feasible: boolean };
  cost: { byRole: Record<Role, number>; allocationM: Record<Role, number>; launchM: number; opsM: number; delayM:
      number; totalM: number; budgetM: number; reservePct: number; lines: Line[] };
  time: { devMonths: number; launchMonth: number; windowMonths: number; delayMonths: number; transferMonths:
      number; lifetimeYears: number };
  pointing: { adcsDeg: number; perInstrument: { id: string; reqDeg: number; quality: number }[] };
}
export interface Check { id: string; severity: Severity; role: Role; title: string; message: string; fixHint:
      string; conceptId: string }
export interface SimInput { scenarioId: ScenarioId; difficulty: DifficultyId; seed: number; design: Design;
      split: BudgetSplit }
export interface PendingEvent { index: number; day: number; event: GameEvent; options: (EventOption & { disabled:
      boolean; disabledReason?: string })[]; autoText?: string }
export interface Sample { day: number; generationW: number; loadW: number; duty: number; stored: number;
      dataGeneratedCum: number; dataReturnedCum: number; scienceCum: number; costCum: number }
export interface EventLogEntry { index: number; day: number; eventId: string; title: string; optionId: string |
      null; byRole: Role; bad: boolean; text: string }
export interface SimResult {
  outcome: Outcome; outcomeReason: string; endDay: number; plannedDays: number; achievedYears: number;
  dataGenerated: number; dataReturned: number; dataLost: number; scienceGenerated: number; scienceReturned:
      number;
  extraCostM: number; finalTotalCostM: number; samples: Sample[]; events: EventLogEntry[];
}
export interface Simulation {
  readonly day: number; readonly pendingEvent: PendingEvent | null; readonly done: boolean;
  step(days: number): void;                // advances; stops early at an event or the end
  decide(optionId: string): void;          // resolves pendingEvent
  current(): Sample;                       // latest values for the live dashboard
  result(): SimResult;                     // valid when done
}
export interface Insight { kind: 'good' | 'bad'; role: Role; text: string; conceptId: string }
export interface ScoreReport {
  total: number; stars: 0 | 1 | 2 | 3; outcomeClass: OutcomeClass; rankTitle: string;
  parts: { objectives: number; science: number; efficiency: number; operations: number; teamwork: number };
  roleGoals: Record<Role, { met: boolean; text: string }>;
  insights: Insight[]; tryNext: string[];
}
export interface TeamContext { reviewDone: boolean; failChecksAtLaunch: number }
export interface Maturity { cml: 2 | 3 | 4; name: 'Initial Feasibility' | 'Trade Space' | 'Point Design';
      greenPct: number; green: number; yellow: number; red: number }
export type SweepParam = 'lifetimeYears' | 'solarArrayW' | 'batteryWh' | 'deltaVMarginPct' | 'busId' | 'radioId'
      | 'groundNetworkId' | 'launchVehicleId' | 'adcsId';
export interface SweepPoint { value: number | string; label: string; scienceEst: number; scienceEstPctOfTarget:
      number; totalCostM: number; launchMarginPct: number | null; powerMarginPct: number; returnPct: number;
      failCount: number; isCurrent: boolean }
```

A14.2 Core API (`src/core/index.ts`, pure functions, no React, no Firebase)

```
export function getCatalog(): Catalog;      // { orbits, launchVehicles, components, instruments, scenarios,
      difficulties, events, glossary, checksText, dialog, characters, constants }
export function defaultDesign(scenarioId: ScenarioId, difficulty: DifficultyId): Design;   // a valid, slightly
      imperfect starter design
export const DEFAULT_SPLIT: BudgetSplit;    // { planner: 35, engineer: 30, science: 25, comms: 10 }
export function computeBudgets(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split:
      BudgetSplit): Budgets;
export function runChecks(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split: BudgetSplit):
      Check[];
export function isUnlocked(itemId: string, difficulty: DifficultyId, scenarioId: ScenarioId): boolean;
   // true if: item tier is basic, OR difficulty.advancedUnlocked, OR scenario is sandbox,
   //          OR the item is an instrument whose tags satisfy one of the scenario's required groups
```

```
export function createSimulation(input: SimInput): Simulation;
export function simulateAll(input: SimInput, decisions: Record<number, string>): SimResult;   // headless replay
export function scoreMission(input: SimInput, result: SimResult, team: TeamContext): ScoreReport;
export function encodeDesign(p: SharePayload): string;  export function decodeDesign(code: string): SharePayload
      | null;
export function encodeChallenge(s: ScenarioId, d: DifficultyId, seed: number): string;  export function
      decodeChallenge(code: string): { scenarioId: ScenarioId; difficulty: DifficultyId; seed: number } | null;
export function newSeed(): number;
export function conceptMaturity(checks: Check[]): Maturity;                                    // F-35
export function findMissionTwin(design: Design, scenarioId: ScenarioId): MissionTwin | null;  // F-33
export function estimateScience(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split:
      BudgetSplit): number;   // no events
export function tradeSweep(design: Design, scenarioId: ScenarioId, difficulty: DifficultyId, split: BudgetSplit,
      param: SweepParam): SweepPoint[];  // F-34
// SharePayload = { v: 1; scenarioId; difficulty; seed; teamName: string; design: Design; split: BudgetSplit;
      decisions: Record<number, string> }
```

A14.3 Room API (src/net/roomApi.ts)

```
export interface PlayerState { uid: string; name: string; characterId: string; connected: boolean; joinedAt:
      number; lastSeen: number }
export interface FundsRequest { id: string; fromRole: Role; toRole: Role; amountM: number; reason: string;
      status: 'pending' | 'accepted' | 'declined'; createdAt: number }
export interface ChatMessage { id: string; uid: string; name: string; text: string; at: number }
export interface RoomMeta { code: string | null; hostUid: string; createdAt: number; scenarioId: ScenarioId;
      difficulty: DifficultyId; seed: number; phase: Phase; teamName: string; version: 1; opsSpeed?: 1 | 2 | 4 |
      8; opsPaused?: boolean }
export interface RoomState {
  meta: RoomMeta; players: Record<string, PlayerState>; roles: Record<Role, string | null>;
  council: { split: BudgetSplit; agreed: Record<string, boolean> }; requests: FundsRequest[];
  design: Design; signoffs: Record<Role, boolean>; reviewDone: boolean;
  decisions: Record<number, { optionId: string; byUid: string; at: number }>;
  suggestions: Record<number, Record<string, string>>; chat: ChatMessage[];
  result: { score: number; stars: number; outcome: string; designCode: string; at: number } | null;
}
export interface RoomApi {
  readonly mode: 'local' | 'online';
  readonly myUid: string;
  subscribe(cb: (s: RoomState) => void): () => void;   // called immediately with current state, then on every
      change
  getState(): RoomState;
  setMeta(patch: Partial<Pick<RoomMeta, 'scenarioId' | 'difficulty' | 'phase' | 'teamName' | 'seed' | 'opsSpeed'
      | 'opsPaused'>>): Promise<void>;  // host only
  updateMe(patch: Partial<Pick<PlayerState, 'name' | 'characterId'>>): Promise<void>;
  claimRole(role: Role): Promise<boolean>;
  releaseRole(role: Role): Promise<void>;
  setSplit(split: BudgetSplit): Promise<void>;          // clears all agreed
  setAgreed(agreed: boolean): Promise<void>;
  requestFunds(fromRole: Role, toRole: Role, amountM: number, reason: string): Promise<void>;
  answerRequest(id: string, accept: boolean): Promise<void>;   // accept moves split points
  updateDesign<R extends Role>(role: R, patch: Partial<Design[R]>): Promise<void>;   // also clears all signoffs
  setSignoff(role: Role, value: boolean): Promise<void>;
  setReviewDone(value: boolean): Promise<void>;
  submitDecision(eventIndex: number, optionId: string): Promise<void>;   // first write wins
  suggest(eventIndex: number, optionId: string): Promise<void>;
  sendChat(text: string): Promise<void>;
  setResult(r: NonNullable<RoomState['result']>): Promise<void>;
  leave(): Promise<void>;
}
export function createLocalRoom(opts: { name: string; characterId: string; scenarioId: ScenarioId; difficulty:
      DifficultyId; seed?: number }): RoomApi;
export function createOnlineRoom(opts: { name: string; characterId: string; scenarioId: ScenarioId; difficulty:
      DifficultyId; seed?: number }): Promise<RoomApi>;   // generates code
export function joinOnlineRoom(code: string, opts: { name: string; characterId: string }): Promise<RoomApi>;   //
      throws 'ROOM_NOT_FOUND' | 'ROOM_FULL' | 'NETWORK'
export function myRoles(s: RoomState, uid: string): Role[];
export function isHost(s: RoomState, uid: string): boolean;
```

A14.4 UI store (src/state/store.ts, Programmer B) — shape all screens use

```
interface GameStore {
  room: RoomApi | null; state: RoomState | null;
  budgets: Budgets | null; checks: Check[];                 // recomputed whenever state.design/split changes
  sim: Simulation | null; simSpeed: 1 | 2 | 4 | 8; paused: boolean;
  score: ScoreReport | null; attempts: { at: number; score: number; designCode: string }[];
  settings: { volume: number; muted: boolean; textScale: 1 | 1.25 | 1.5; reducedMotion: boolean };
  startSolo(o): void; createRoom(o): Promise<void>; joinRoom(code, o): Promise<void>; leaveRoom(): Promise<void>;
}
```

A15. Art direction and the complete asset manifest

Style: clean, readable pixel art; NASA-mission-control mood with warm Egyptian sunlight accents. Light comes from the **top-left**. 1-pixel dark outline (#181425) on characters and objects, selective (lighter outline colour where an edge faces the light). Maximum 3 shades per material + 1 highlight. No anti-aliasing, no blur, no gradients except dithering. Every PNG is drawn at **1× native size** and exported as PNG-32 with transparency; the game scales it ×2, ×3 or ×4 with nearest-neighbour filtering. **Never** export upscaled images.

Palette — ENDESGA 32 (use only these 32 colours):

```
#be4a2f #d77643 #ead4aa #e4a672 #b86f50 #733e39 #3e2731 #a22633
#e43b44 #f77622 #feae34 #fee761 #63c74d #3e8948 #265c42 #193c3e
#124e89 #0099db #2ce8f5 #ffffff #c0cbdc #8b9bb4 #5a6988 #3a4466
#262b44 #181425 #ff0044 #68386c #b55088 #f6757a #e8b796 #c28569
```

UI colour mapping (CSS variables, Programmer B): background #181425 · panel #262b44 · panel border #3a4466 · text #ffffff · muted text #c0cbdc / #8b9bb4 · accent gold #feae34 · primary blue #0099db · pass #63c74d · warn #f77622 · fail #e43b44 · role planner #feae34 · engineer #f77622 · science #63c74d · comms #2ce8f5. Status is never shown by colour alone: pass = check mark, warn = triangle "!", fail = cross.

Skin tones from the palette: light #e8b796 (shade #c28569), medium #c28569 (shade #b86f50), brown #b86f50 (shade #733e39), dark #733e39 (shade #3e2731). Never use pure black for skin shading.

A15.1 Crew roster (`src/data/characters.json`)

id	name	from	game role	playable	appearance summary
nour	Nour El-Sayed	Egypt	planner	yes	medium skin; navy hijab (#124e89) with gold edge, neatly wrapped; calm confident smile; ~35
lucas	Lucas Moreau	France	planner	yes	light skin; short wavy brown hair; light stubble; round glasses; ~40
diaa	Diaa El Awdi	Egypt	planner	yes	a real person (appears with his permission); light-olive skin; short salt-and-pepper hair combed back (grey sides, darker top); dark eyebrows; wide warm smile; clean-shaven; signature navy suit, light-blue shirt, silver tie, gold watch; ~50s
omar	Omar Farouk	Egypt	engineer	yes	brown skin; short black curly hair; short beard; rolled sleeves; ~30
elena	Elena Petrova	Russia	engineer	yes	light skin with freckles; blonde hair in a bun; focused look; ~45
amara	Amara Okafor	Nigeria	science	yes	dark skin; natural coily hair in two puffs; gold hoop earrings; holds tablet; ~32
hiroshi	Hiroshi Tanaka	Japan	science	yes	light skin; straight black hair with grey streaks; field notebook; ~55
kwame	Kwame Mensah	Ghana	comms	yes	dark skin; short fade haircut; headset with mic; ~28
layla	Layla Al-Mansouri	UAE	comms	yes	medium skin; long dark ponytail; headset; bright smile; ~27
hassan	Dr. Hassan Nasr	Egypt	flight director (NPC, tutorial guide)	no	brown skin; short grey beard; glasses; mission vest; warm smile; ~60
priya	Priya Nair	India	ground station operator (NPC, comms tips)	no	brown skin; long dark braid; headset; uses a wheelchair; ~34

Every crew member wears a white flight suit (#ffffff / #c0cbdc / #8b9bb4) with a shoulder stripe and chest patch in their **role colour**. Exception: Diaa El Awdi wears his signature navy suit (with a gold Commander pin on the lapel) in his portraits and idle sprite; his suit sprite is the standard spacesuit. NPCs wear the mission control vest in #3a4466 with a gold badge. All characters are adults.

A15.2 Asset manifest (exact paths under `public/assets/`)

Programmer B generates a coloured placeholder for every row at H1 (`scripts/make-placeholders.mjs`). Artists overwrite the same path with the final art: the game picks it up with no code change. Character art = **C**, everything else = **E**.

Path	Size (px, 1×)	Frames	Owner	Content
characters/portrait_{id}_{expr}.png	64×64	1	C	for each of 11 ids × expr ∈ neutral, happy, worried (33 files); head and shoulders, transparent background

Path	Size (px, 1×)	Frames	Owner	Content
characters/sprite_{id}_idle.png	128×48	4 × (32×48)	C	11 files; standing idle (breathing/blink loop), flight suit, no helmet; facing right-front
characters/sprite_{id}_suit.png	64×48	2 × (32×48)	C	9 playable only; full spacesuit with helmet visor up (face visible); frame 2 = waving
bg/bg_title.png	480×270	1	E	Earth's limb, stars, a rocket climbing, logo space in upper third left clear
bg/bg_mission_control.png	480×270	1	E	mission control room, big wall screen, consoles; lower 90 px calm for UI
bg/bg_clean_room.png	480×270	1	E	spacecraft assembly clean room; centre area empty (spacecraft preview drawn there)
bg/bg_launch_pad.png	480×270	1	E	coastal desert launch site at dawn; launch tower right of centre; rocket drawn by code at x=240
bg/bg_space_tile.png	64×64	1	E	seamless tiling starfield
planets/planet_earth.png	128×128	1	E	Earth with Africa/Nile visible
planets/planet_moon.png	96×96	1	E	Moon, south pole craters visible
planets/planet_mars.png	112×112	1	E	Mars
planets/planet_sun.png	64×64	1	E	Sun disc with glow (dithered)
lv/lv_spark.png	16×64	1	E	small rocket
lv/lv_arrow.png	24×96	1	E	light-medium rocket
lv/lv_workhorse.png	24×128	1	E	medium rocket (also used for rideshare)
lv/lv_titan.png	48×128	1	E	heavy rocket with side boosters
fx/fx_flame.png	64×24	4 × (16×24)	E	engine flame loop (scaled to rocket width by code)
fx/fx_smoke.png	192×32	6 × (32×32)	E	launch smoke puff
fx/fx_explosion.png	512×64	8 × (64×64)	E	launch failure explosion (cartoonish, not graphic)
sc/bus_cube.png · bus_small.png · bus_medium.png · bus_large.png	16×16 · 32×32 · 48×48 · 64×64	1	E	spacecraft body, side view, gold foil + panels
sc/sa_si_panel.png · sc/sa_gaas_panel.png	16×24	1	E	one solar panel (silicon = blue #124e89, GaAs = dark #262b44 with violet glint)
sc/rtg.png	8×16	1	E	finned cylinder
sc/hga.png	24×24	1	E	white dish antenna
sc/antenna_lga.png	8×8	1	E	small stub antenna
sc/thruster.png	8×8	1	E	nozzle
sc/sc_marker.png	8×8	1	E	tiny spacecraft for the orbit view
items/{id}.png	32×32	1	E	one icon per catalog id (list A15.3)
icons/{id}.png	16×16	1	E	res_mass res_power res_budget res_data res_comms res_time res_deltav res_science st_pass st_warn st_fail st_info role_planner role_engineer role_science role_comms ui_star_full ui_star_empty ui_lock ui_help
events/ev_{eventId}.png	96×64	1	E	illustration per event (10 files)
scenarios/sc_{scenarioId}.png	160×90	1	E	scenario card art (6 files)
ui/ui_panel.png · ui_panel_dark.png	24×24	1	E	9-slice frame, 8 px corners
ui/ui_button.png · _hover · _pressed · _disabled	24×24	1	E	9-slice button, 8 px corners
ui/ui_tab.png · ui_tab_active.png	24×24	1	E	9-slice tab
ui/logo_deltav.png	256×64	1	E	"DELTA-V TYCOON" pixel logotype
ui/favicon.png	32×32	1	E	rocket icon
ui/og_image.png	1200×630	1	E	link preview (logo + crew lineup); may be composed from other art
audio/music_menu.mp3 · music_ops.mp3	loop	–	E	calm synth loops, CC0 or CC-BY

Path	Size (px, 1×)	Frames	Owner	Content
audio/sfx_click · sfx_hover · sfx_success · sfx_fail · sfx_warning · sfx_launch · sfx_event · sfx_signoff · sfx_coin .mp3	short	–	E	sound effects, CC0 or CC-BY

A15.3 `items/` icon ids (32×32 each, 52 files)

Buses: bus_cube bus_small bus_medium bus_large · Power: sa_si sa_gaas rtg bat_liion bat_liion_adv · Computers: obc_basic obc_adv · ADCS: adcs_mag adcs_wheels adcs_precision · Propulsion: prop_none prop_coldgas prop_mono prop_biprop prop_electric · Comms: radio_uhf radio_sband radio_xband radio_kaband hga · Ground: gs_campus gs_commercial gs_deepspace · Vehicles: lv_spark lv_rideshare lv_arrow lv_workhorse lv_titan · Orbits: orbit_leo orbit_sso orbit_geo orbit_lunar orbit_l2 orbit_mars · Instruments: cam_wide cam_hires imager_ms imager_tir radar_sar sounder_atm lightning_map spec_neutron lidar_alt magnetometer spec_atm det_particle relay_uhf tel_ir.

Spacecraft preview composition (Programmer B draws it; artists keep these anchors in mind): canvas 160×120 (1×). Bus centred at (80,60). Solar panels attach at the bus left/right edge centre; panel count per side = `clamp(ceil(solarArrayW / 500), 0, 6)`, laid side by side outward. HGA on top centre; low-gain antenna on top-right corner; RTGs on bottom-left; thruster at bottom centre; instruments drawn as their 32×32 item icons scaled to 16×16 on the front face in a row, max 6.

A16. Timeline, communication and definition of done

All times are Cairo time. H0 = Friday 1:00 PM.

When	Milestone (owner)
Before H0 (allowed)	Accounts: GitHub (all), Firebase project (A), itch.io optional (B). Install Node 20, VS Code, Aseprite/LibreSprite/Piskel. Join event Discord, choose mentor tracks. Create EMPTY public repo (L). Read this whole document.
H0–H0:30	Kickoff: Lead walks everyone through Part A. Challenge resources (released at kickoff) reviewed by Lead.
H1	Repo scaffold + placeholders + deploy workflow pushed (B). Palette + style test: 1 portrait (C) and 3 icons (E) shown to Lead.
H1:30	`src/core/types.ts` pushed (A).
H2	All `src/data/*.json` v1 pushed (L). Test plan skeleton (Q).
H4	`computeBudgets` + `runChecks` + `createLocalRoom` working (A). Title → solo → design screen with live meters (B).
H6 (7 PM)	**First public build online.** Solo: design → review → launch → ops (no events yet) → report. QA starts build testing.
H9 (10 PM)	Events + scoring + insights + Mission Twin + Concept Maturity done (A). All P0 screens incl. review meter, report, briefing with requirements (B). Online create/join working (A). Playable portraits + item icons in (C, E).
H11 (12 AM)	Online lobby, council, sign-offs, decisions working (A+B). Trade-Space Explorer, tutorial, Academy (A+B+L). All art in (C, E).
H12 (1 AM)	**Online go/no-go test (Q). Cutoff rule:** if online rooms are not stable, the title screen shows "Team Mode (same screen)" and the online buttons are hidden behind `?online=1`.
H13 (2 AM)	Balance pass (L+A), P1 polish, text polish.
H14 (3 AM)	**FEATURE FREEZE.** Bug fixes only. Tag `golden-1` on the last good build.
H14–H16 (3–5 AM)	P0 bug fixes (A, B); regression pass (Q); footage capture (Q+L). Sleep shifts begin: max 2 people asleep at a time.
H16–H21 (5–10 AM)	Video edit + slides (L, help from C/E), README + summary + sources (L). Only P0 fixes. Tag `golden-final` by 10 AM.
H22–H23 (11 AM–12 PM)	**Submit** local form, then global site (L). QA verifies every link in a private window.
H24 (1 PM)	Official deadline. Then rest; prepare for People's Choice at ~5 PM.

Mentor tracks (each member picks a different one on the event Discord): Lead → Storytelling & Video Editing · Programmer A → Software Engineering · Programmer B → Web Development · Character artist → Art & Visualization · Environment/asset artist → Game Development · QA → UI/UX Design.

Discord channels (team server or a private group): `#general` (coordination), `#bible-changes` (Lead only), `#builds` (B posts every deploy URL + version), `#bugs` (Q; mirrors GitHub Issues), `#art-drops` (C/E post previews), `#help` (blocked? post here immediately). **Status ping every 2 hours** from every member: done / doing / blocked.

Definition of done (for any task): it works in the deployed public build, not only on your laptop; it uses the exact IDs and paths of this document; it has no console errors; QA has seen it once.

A17. Reference designs (expected values — tests and QA use these)

Computed with the formulas in A9 (rounded). If the code gives different numbers, the code is wrong (or A9 was changed — check `#bible-changes`).

Scenario	Design	Total kg	Launch margin	Load W	Gen W	Power margin	Battery req Wh	Downlink Gbit/day	Return	Cost $M
nile_watch	SSO 3 yr, Spark, 10% margin, bus_small, sa_si 450 W, bat_liion 350 Wh, obc_basic, adcs_wheels, prop_mono, imager_ms + imager_tir, radio_xband, campus	185.7	7.1%	189.8	248.6	31.0%	277	216.0	100%	70.8
storm_watch	GEO 5 yr, Workhorse, 10%, bus_medium, sa_si 700 W, bat_liion 1500 Wh, obc_basic, adcs_wheels, prop_biprop, sounder_atm + imager_tir + lightning_map, radio_xband, commercial	1005.8	81.7%	474.0	589.0	24.3%	1422	1944.0	100%	219.7
ice_hunter	Lunar 1.5 yr, Arrow, 10%, bus_medium, sa_si 700 W, bat_liion 600 Wh, obc_basic, adcs_wheels, prop_biprop, spec_neutron + lidar_alt + imager_tir, radio_xband, deepspace	626.4	10.5%	299.0	357.0	19.4%	586	259.2	100%	143.8
red_relay	Mars 2 yr, Workhorse, 10%, bus_medium, sa_gaas 1800 W, bat_liion 600 Wh, obc_adv, adcs_wheels, prop_biprop, spec_atm + relay_uhf + cam_wide, radio_xband + hga, deepspace	705.8	71.8%	353.2	427.6	21.1%	589	38.9	100%	194.7
first_light	L2 5 yr, Workhorse, 10%, bus_medium, sa_gaas 750 W, bat_liion 100 Wh, obc_basic, adcs_precision, prop_mono, tel_ir, radio_kaband, deepspace	755.1	74.8%	520.7	637.5	22.4%	50	77.8	100%	487.7
(bad example)	LEO 2 yr, Spark, bus_cube, sa_si 100 W, bat 50 Wh, obc_basic, adcs_mag, prop_none, cam_wide, radio_sband, campus	21.8	92.7%	30.4	52.7	73.3%	50	3.6	18%	18.9

The bad example is deliberately cheap: it launches fine but returns only 18% of its data, and the magnetorquers blur the camera (quality 0.2). The report must explain both. **Several different designs win each scenario** (e.g. ice_hunter on Arrow at 10.5% margin vs Workhorse at 82%: cheaper but riskier). That is the "no single perfect solution" point judges look for.